VELS
INSTITUTE OF SCIENCE, TECHNOLOGY & ADVANCED STUDIES (VISTAS)
(Deemed to be University Estd. u/s 3 of the UGC Act, 1956)
PALLAVARAM, THALAMBUR, PERIYAPALAYAM, THIRUVANMIYUR - CHENNAI

AI-BASED SMART EXPENSE MANAGEMENT SYSTEM WITH AUTOMATED CATEGORIZATION AND BUDGET RECOMMENDATIONS

MINI PROJECT REPORT

Submitted to the

SCHOOL OF ENGINEERING
DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY
IN
ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

KOTAKONDA SHRAVAN B – [24619118]
THAHSEEN BASHA S – [24619132]
VARUN SEKAR V – [22619184]

VELS INSTITUTE OF SCIENCE, TECHNOLOGY AND ADVANCED STUDIES (VISTAS)
CHENNAI – 600 117
OCTOBER 2026

BONAFIDE CERTIFICATE

This project report entitled "AI-Based Smart Expense Management System with Automated Categorization and Budget Recommendations" is hereby certified to be the bonafide project of KOTAKONDA SHRAVAN B – [24619118], THAHSEEN BASHA S – [24619132] and VARUN SEKAR V – [22619184], conducted under my guidance. It is further certified that, to the best of my knowledge, the project undertaken by them has not been undertaken elsewhere, nor used to obtain a degree/diploma/certificate from any other University/Institution on any other occasion.

SIGNATURE OF THE HOD
Dr. K. Ulaga Priya, M.E., Ph.D
Associate Professor & Head
Department of AI&ML
School of Engineering VISTAS,
Chennai

SIGNATURE OF THE SUPERVISOR
Mrs. Janani. P, M.Tech
Assistant Professor
Department of AI&ML
School of Engineering
VISTAS, Chennai

Submitted for the Project Work and Viva-Voce examination held on …… at Vels Institute of Science, Technology and Advanced Studies (VISTAS), Chennai – 600 117.

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide, Mrs. P. Janani, for her guidance, encouragement and continuous support during the planning, development, testing and documentation of the project.

We thank the Head of the Department and the faculty members of the Department of Artificial Intelligence and Machine Learning, School of Engineering, Vels Institute of Science, Technology and Advanced Studies, for providing the academic environment and facilities required to carry out the work.

We acknowledge the Android and Kotlin documentation and the research works referenced in this report. These materials helped us understand notification-listener services, local storage, text classification, regression forecasting and evaluation measures.

Finally, we thank our families, friends and classmates for their encouragement and support throughout the project.

KOTAKONDA SHRAVAN B – [24619118]
THAHSEEN BASHA S – [24619132]
VARUN SEKAR V – [22619184]

ABSTRACT

Personal expense management requires a user to record transactions, organise them into categories, review historical spending and make informed plans for future expenditure. This project develops an Android expense-tracking application with supported notification-based transaction capture, manual expense entry, local transaction storage, merchant-based categorisation, monthly expenditure summaries and Linear Regression-based next-month estimation.

The application uses Kotlin for application logic, Jetpack Compose for the interface, SQLite for local persistence and Android NotificationListenerService for supported notification events after the user grants access. The listener filters likely outgoing-payment notifications, the transaction parser attempts to extract an amount and merchant, duplicate checking compares recent merchant-and-amount matches, and saved expenses can be reviewed in transaction history and monthly summaries.

The runtime class named NaiveBayesClassifier does not currently train or evaluate a probabilistic Naive Bayes model. It uses a predefined merchant-keyword map, chooses the longest matching keyword and returns the associated category or Other. LinearRegressionPredictor estimates the next monthly total from historical monthly totals, uses the last value as a one-month baseline, and returns no prediction when there is no history. The supplied files do not include measured classification or forecasting metrics, so this report does not claim numerical accuracy.

Implementation-alignment note: although “Budget Recommendations” appears in the project title and an earlier report screenshot shows a budget screen, the Kotlin source ZIP supplied for this revision contains no monthly-income input, saved-income field or budget-recommendation calculation/screen. That feature is documented as planned scope and is not counted as implemented or tested in this version.

Keywords: Android application, expense management, merchant-keyword categorisation, Linear Regression, SQLite, Kotlin, Jetpack Compose, notification processing, local persistence.

TABLE OF CONTENTS

ABSTRACT iv

LIST OF FIGURES 1

LIST OF TABLES 2

ABBREVIATIONS 3

CHAPTER 1 – INTRODUCTION 4

1.1 Personal Expense Management 4

1.2 Background and Need for Automation 5

1.3 Problem Statement 6

1.4 Motivation and Objectives 7

1.5 Scope and Boundaries 8

1.6 Contributions of the Integrated Application 9

1.7 Importance of Privacy and Responsible Automation 10

1.8 Organisation of the Report 11

CHAPTER 2 – RELATED WORK 12

2.1 Transaction Text Classification 12

2.2 Naive Bayes for Text Classification 13

2.3 Regression and Spending Forecasting 14

2.4 Mobile Expense Applications and Workflow Comparison 15

2.5 Local Storage, Privacy and Data Integrity 16

2.6 Research Gap and Integrated Design Opportunity 17

2.7 Evaluation Concepts for Classification and Forecasting 18

2.8 Chapter Summary 19

CHAPTER 3 – REQUIREMENTS AND FEASIBILITY 20

3.1 System Requirements Overview 20

3.2 Functional Requirements 21

3.3 Non-Functional Requirements 22

3.4 Intended Users and Use Cases 23

3.5 Technical Feasibility 24

3.6 Operational Feasibility 25

3.7 Economic Feasibility 26

3.8 Assumptions, Dependencies and Constraints 27

CHAPTER 4 – SYSTEM ANALYSIS AND DESIGN 28
4.1 Overall System Architecture 28
4.2 Module Design and Responsibilities 29
4.3 End-to-End Data Flow 30
4.4 Local Database Design 31
4.5 Data Dictionary and Integrity Rules 32
4.6 Dashboard and User Interface Design 33
4.7 Validation, Error Handling and Recovery 34
4.8 Budget Recommendation Design 35
4.9 Security and Privacy Design 36
CHAPTER 5 – METHODOLOGY AND ALGORITHMS 37
5.1 Processing Methodology Overview 37
5.2 Notification Capture and Permission Process 38
5.3 Transaction Text Parsing and Validation 39
5.4 Duplicate Transaction Detection 40
5.5 Multinomial Naive Bayes Formulation 41
5.6 Current Merchant-Keyword Categorisation 42
5.7 Linear Regression Formulation 43
5.8 Monthly Expenditure Aggregation 44
5.9 Interpretation and Responsible Display of Predictions 45
5.10 Algorithm Limitations and Future Improvements 46
CHAPTER 6 – IMPLEMENTATION 47
6.1 Development Environment and Technology Stack 47
6.2 Kotlin Application Logic and Module Integration 48
6.3 User Interface with Jetpack Compose 49
6.4 SQLite Local Persistence 50
6.5 Notification Access and User Consent 51
6.6 Manual Expense Entry 52
6.7 Automated Category Assignment 53
6.8 Transaction History and Monthly Summaries 54
6.9 Spending Prediction Display 55
6.10 Budget Recommendation Implementation 56

CHAPTER 7 – TESTING AND RESULTS 57
7.1 Functional Testing Strategy 57
7.2 Test Environment and Test Data Principles 58
7.3 Functional Test Cases and Expected Outcomes 59
7.4 Boundary and Error Testing 60
7.5 Usability-Oriented Checks 61
7.6 Results Supported by the Prototype 62
7.7 Future Quantitative Evaluation Plan 63
CHAPTER 8 – CONCLUSION AND FUTURE SCOPE 64
8.1 Conclusion 64
8.2 Project Contributions 65
8.3 Limitations 66
8.4 Future Enhancements 67
8.5 Final Summary 68
REFERENCES 69
APPENDIX A – APPLICATION SCREENSHOTS 71
APPENDIX B – IMPORTANT CODE EXTRACTS 76

LIST OF FIGURES

Figure 1. High-level system architecture 28

Figure 2. Notification-to-storage transaction lifecycle 30

Figure 3. Conceptual transaction record structure 32

Figure 4. Application navigation and main dashboard functions 33

Figure 5. Supported notification and manual-entry paths 38

Figure 6. Proposed Multinomial Naive Bayes classification workflow 41

Figure 7. Monthly expenditure forecasting workflow 43

Figure 8. Functional testing workflow 57

Figure 9. Home dashboard and expense totals 71

Figure 10. Dashboard summary and monthly spending 72

Figure 11. Transaction history screen 73

Figure 12. Manual Add Expense screen 74

Figure 13. Budget Recommendation screen 75

LIST OF TABLES

Table 1. Core application components and responsibilities 20

Table 2. Functional requirements 21

Table 3. Non-functional requirements 22

Table 4. User use cases 23

Table 5. Application module responsibilities 29

Table 6. Transaction data dictionary 32

Table 7. Functional test cases 59

Table 8. Boundary and error tests 60

Table 9. Prototype evidence and limitations 62

Table 10. Proposed quantitative evaluation measures 63

ABBREVIATIONS

TERM	FULL FORM	USE IN PROJECT
AI	Artificial Intelligence	Field concerned with systems performing tasks associated with intelligent behaviour.
API	Application Programming Interface	Defined interface used by software components to interact.
DB	Database	Organised collection used to store and retrieve transaction records.
UI	User Interface	Screens and controls through which the user interacts with the application.
UPI	Unified Payments Interface	Digital payment system whose notifications may provide transaction details.
SQL	Structured Query Language	Language used to manage relational database data.
SQLite	SQLite database engine	Lightweight local relational database used by the application.
NLP	Natural Language Processing	Techniques for processing and interpreting text.
NB	Naive Bayes	Probabilistic classification family described as the intended category model.
LR	Linear Regression	Regression method used to estimate next-month spending.
MAE	Mean Absolute Error	Average absolute difference between predicted and observed values.
RMSE	Root Mean Squared Error	Square root of mean squared prediction error.
F1-score	F1-score	Harmonic-mean measure combining precision and recall.
SDK	Software Development Kit	Collection of tools and libraries used for software development.
APK	Android Package	Installable package format for Android applications.
CRUD	Create, Read, Update, Delete	Common operations performed on stored data.

CHAPTER 1 – INTRODUCTION

1.1 Personal Expense Management

Personal expense management is the systematic process of recording, organising, reviewing and interpreting money spent over a period. In the context of this project, an expense record contains information such as merchant name, amount, category and transaction time. These fields provide a compact representation of spending activity and support later aggregation. A user can examine individual transactions to understand what was purchased and can also examine monthly totals to understand the overall level of expenditure.

Digital payments make transaction information more readily available to the user, but availability does not automatically produce a useful expense record. Payment notifications can contain merchant names and amounts, yet the wording differs between applications and may contain balances, reference numbers or other information. The project therefore treats notification processing as a supported input path rather than as a universal financial data feed. The manual-entry path remains necessary for unsupported payments, cash expenses, missing notifications and cases in which parsing cannot reliably identify the required fields.

The proposed application adds interpretation to transaction storage. Categorisation groups related expenses, monthly summaries provide a time-based view and the forecasting component uses historical monthly totals to estimate a future amount. The budget recommendation screen adds an income-informed view of spending. These functions are intended to reduce the effort required to maintain a useful personal record while keeping the core data on the device.

The system should be understood as an applied software project rather than a complete financial management platform. It does not connect directly to bank systems, does not guarantee that every payment will be captured and does not claim that a forecast is always accurate. The source material explicitly separates functional prototype behaviour from formal machine-learning evaluation. The expanded report follows that distinction and focuses on explaining how the implemented workflow operates and how it can be evaluated in future work.

In the supplied source, Expense.kt defines the shared record fields as merchant, amount, category, date and time. DatabaseHelper reads the timestamp from SQLite and formats it for presentation. This common record structure allows notifications and manual entries to appear together in the same transaction history.

1.2 Background and Need for Automation

Traditional expense tracking often follows a manual cycle: the user notices a purchase, opens an application, enters an amount and merchant, selects a category and saves the record. The method is transparent and does not depend on external notification formats, but it relies heavily on user discipline. When several purchases occur during a busy day, the probability of a missed entry can increase. Missing records affect both the visible transaction history and every later calculation derived from that history.

Notification-based capture can reduce repeated data entry when a payment application exposes a readable transaction notification. The application can observe the notification event after the user grants notification access, identify a plausible amount and merchant, and then pass the record through validation and duplicate checking. This is useful because the user does not need to type every supported transaction manually. At the same time, notification access is sensitive and the system must process information only for the intended expense-tracking purpose.

Automation also introduces new reliability questions. Different payment applications may use different phrases, punctuation, currencies, number formats and merchant descriptions. Some notifications may be redacted or may contain several numbers. Therefore, the parser should validate its results and avoid creating an expense when the evidence is ambiguous. The project addresses this by retaining manual entry as a recovery path.

From a user perspective, automation is most valuable when it reduces friction without hiding what the system has done. The interface should allow users to review stored transactions, distinguish actual spending from estimates and recognise that category assignment is based on the current implementation. Clear presentation and explicit limitations are therefore part of the design, not merely documentation after implementation.

The current listener first checks that a notification contains a currency amount and a payment-related phrase, then rejects common non-expense cases such as received funds, refunds, cashback and offers. This filtering reduces obvious false positives but cannot guarantee that every payment provider message will be recognised.

1.3 Problem Statement and Project Challenges

The central problem addressed by the project is the difficulty of maintaining a complete and useful personal expense history when transaction recording depends primarily on manual effort. Three related challenges motivate the system. First, transaction capture must be convenient enough to encourage consistent recording. Second, raw transaction descriptions need to be organised into meaningful spending categories. Third, historical records should provide more than a list; they should support monthly summaries, a future spending estimate and budget-oriented guidance.

The first challenge is input uncertainty. A notification may contain the desired information, may contain partial information or may not appear at all. The system cannot assume that every payment provider follows the same message template. A robust workflow therefore has to treat notification capture as opportunistic and manual entry as a supported alternative.

The second challenge is classification. Merchant names can be informative, but they can also be ambiguous. A fixed keyword mapping is simple and predictable for configured merchants, yet it cannot learn new patterns. The project material proposes Multinomial Naive Bayes as a probabilistic approach, but the supplied runtime implementation currently performs deterministic keyword matching. Documenting this difference is essential because it affects what can legitimately be claimed about the system.

The third challenge is forecasting. A sequence of monthly totals can show a trend, but personal expenditure can change suddenly. Linear Regression is easy to implement and interpret, but it is only an approximation. The system should therefore communicate a forecast as an estimate and should avoid unsupported claims about predictive accuracy until chronological evaluation is performed.

The present boundary is therefore deliberate: the app does not access a bank account directly, does not read a full statement and cannot recover details that an operating system or payment app hides. A rejected or unreadable notification must be entered manually if the user wants it included in totals.

1.4 Motivation and Objectives

The motivation for the project is to build a practical mobile workflow in which transaction capture, organisation and interpretation are connected. A user should not need separate tools for recording an expense, viewing monthly totals and receiving a basic budget-oriented summary. The local-first approach is also motivated by the sensitivity of financial information. Keeping the core records in SQLite avoids making a remote service a requirement for the primary workflow.

The primary objective is to develop an Android application that accepts supported notification-based transactions and manual expenses, validates the information, stores it locally and presents the records in a readable history. A second objective is to reduce repetitive manual entry by extracting merchant and amount information from supported notifications. A third objective is to assign categories automatically. A fourth objective is to aggregate monthly spending and estimate a future monthly amount using the implemented Linear Regression procedure.

The budget recommendation component extends the workflow by allowing a user to enter monthly income and view category-oriented guidance based on stored spending. This is presented as a planning aid rather than financial advice. The system should not imply that a recommendation is optimal or universally appropriate because the quality of the recommendation depends on the completeness and accuracy of the transaction history.

Another objective is maintainability. Notification processing, parsing, categorisation, persistence, prediction and presentation should be separated conceptually so that each component can be improved without rewriting the entire application. This modular approach is particularly important for future work because the current keyword-based category assignment may eventually be replaced or supplemented by a trained classifier.

The delivered code fully supports the capture, persistence, history, categorisation and prediction objectives. Income-based budget recommendations remain a design objective rather than a verified current feature because no matching income storage or screen appears in the supplied ZIP.

1.5 Scope and Boundaries

The scope includes a native Android application, supported notification-based transaction capture, manual expense entry, transaction parsing, validation, duplicate handling, local SQLite persistence, category assignment, transaction history, monthly aggregation, Linear Regression-based expenditure estimation and income-informed budget guidance. These features form the core workflow documented in the supplied project material.

The project does not claim direct access to complete bank statements or private payment-provider APIs. It does not guarantee compatibility with every payment application and does not guarantee that every transaction will generate a readable notification. It also does not claim that a user's complete financial position can be inferred from the transactions stored by the application.

The scope of categorisation is similarly bounded. The current runtime implementation uses predefined merchant keywords. It can provide repeatable results for configured merchants, but it does not learn from labelled transaction examples. The theoretical Naive Bayes formulation is included because it is part of the project methodology and represents a future path toward a trained text classifier.

Forecasting is bounded by the available monthly history. A prediction based on a short or incomplete history may be unstable. The application therefore needs to distinguish recorded totals from model-generated estimates and should require sufficient history before presenting a prediction. Formal accuracy claims are outside the current evidence and are reserved for future evaluation.

For clarity, the feature list separates implemented scope from planned scope. Implemented items in the supplied source include notification filtering/parsing, manual expense entry, duplicate checking, SQLite storage, merchant keyword assignment, monthly totals and Linear Regression prediction. The income-input budget recommendation flow remains pending implementation.

1.6 Contributions of the Integrated Application

The first contribution is an integrated transaction-input workflow. Supported notifications can enter the processing pipeline automatically after permission is granted, while manual entry ensures that the user can record transactions that are not automatically captured. This dual-path design is more practical than assuming that one input method will always succeed.

The second contribution is local transaction organisation. SQLite provides persistent storage for merchant, amount, category and timestamp information. Local persistence allows the application to build a history without requiring a remote database for the main workflow. It also provides a stable source for monthly aggregation and later analysis.

The third contribution is automated category assignment. In the current implementation, the category is derived from predefined merchant keywords. The project documentation also presents Multinomial Naive Bayes as the intended probabilistic model. The report treats the current deterministic implementation and the proposed probabilistic model as separate states of development.

The fourth contribution is lightweight expenditure estimation. Historical transactions are grouped by month and a Linear Regression procedure estimates the next monthly total. The budget recommendation interface then provides a user-facing planning view based on income and spending distribution. Together, these features demonstrate how a small Android application can combine data capture, local analytics and user guidance.

The practical contribution is a connected workflow rather than a new machine-learning algorithm. The code can be inspected file by file: NotificationReaderService.kt receives notification events, TransactionParser.kt extracts fields, NaiveBayesClassifier.kt maps merchant keywords, DatabaseHelper.kt persists records, and LinearRegressionPredictor.kt estimates the next monthly total.

1.7 Importance of Privacy and Responsible Automation

Financial transaction records are sensitive because they can reveal spending habits, merchants and approximate financial behaviour. The project's local-first architecture reduces dependence on remote data transfer for the core transaction workflow. However, local storage alone does not remove all privacy concerns. Notification access can expose notification content from other applications, so the user must explicitly grant access and understand why it is requested.

Responsible automation also requires transparency. When the parser extracts a merchant and amount, the application should avoid silently accepting values that are obviously incomplete. When a category is assigned by a fixed keyword mapping, the user should not be led to believe that the application has learned from a large training dataset. Similarly, a forecast should be labelled as an estimate rather than a guaranteed future expense.

These principles influence design decisions throughout the project. Validation protects data quality, duplicate checking protects totals, manual entry protects coverage and clear labels protect interpretation. The same principles also define the future evaluation plan: quantitative claims should be based on actual labelled data and observed future spending rather than on example values displayed in screenshots.

The privacy and transparency requirements therefore complement the functional requirements. A system that captures transactions automatically but cannot explain its permissions or distinguish estimates from actual records would be less useful even if its basic functions worked. The project aims to demonstrate both practical functionality and responsible presentation.

Privacy controls should be considered across the entire lifecycle of a transaction. The app asks the user to enable notification access through Android settings, but that permission can expose notification text from other apps to the listener. The current workflow uses this text to identify payment details and stores the resulting expense locally; it does not show a remote upload service in the supplied Kotlin files. This should not be interpreted as proof of encryption or complete device security. Future versions should document data retention, backup behaviour and how a user can remove saved records.

1.8 Organisation of the Report

This expanded report is organised into eight chapters followed by references and appendices. Chapter 1 establishes the background, problem, motivation, objectives, scope and contributions. Chapter 2 reviews related concepts including transaction text classification, Naive Bayes, regression forecasting, mobile expense-management workflows, local storage and evaluation principles.

Chapter 3 translates the project goals into functional and non-functional requirements and examines technical, operational and economic feasibility. Chapter 4 describes the architecture, modules, data flow, database structure, interface design, validation mechanisms, budget recommendation design and privacy considerations.

Chapter 5 presents the methodology and algorithms. It explains notification capture, transaction parsing, duplicate handling, the theoretical Multinomial Naive Bayes formulation, the current merchant-keyword implementation, Linear Regression and monthly aggregation. Chapter 6 focuses on implementation using Kotlin, Jetpack Compose, SQLite and Android notification services.

Chapter 7 presents the testing approach and the evidence supported by the prototype. It deliberately separates functional testing from formal machine-learning evaluation and provides a future quantitative evaluation plan. Chapter 8 concludes the report, summarises the contributions and describes future enhancements. References document the main sources identified in the project material, while the appendices provide screenshot and code-oriented supporting material.

This organisation also distinguishes expected design from delivered source code. Screenshots supplied with the report are included as supporting interface references, but a screenshot alone is not treated as evidence that the corresponding behaviour exists in the uploaded implementation.

CHAPTER 2 – RELATED WORK

2.1 Transaction Text Classification

Transaction descriptions are a specialised form of short text. Unlike long documents, they may contain only a few informative tokens, abbreviations, merchant names and numeric values. Nevertheless, the wording can provide useful signals for assigning a spending category. A description containing a food-delivery merchant may suggest Food, while a ride-hailing merchant may suggest Travel. The project treats merchant and transaction text as the main information source for automated category assignment.

Research on text classification demonstrates that probabilistic and discriminative methods can be applied to short textual descriptions when labelled examples are available. The project material refers to transaction-classification research and proposes Multinomial Naive Bayes because it is lightweight and conceptually suitable for token-based text features. The important requirement is not merely selecting an algorithm but creating representative training data and consistent category labels.

The current prototype takes a simpler deterministic approach. It lowercases merchant text, searches for predefined keywords and chooses the longest matching keyword. This approach has an important engineering advantage: its behaviour is easy to understand and reproduce. Its main limitation is coverage. A merchant that is not present in the mapping cannot benefit from learned patterns and may be returned as Other.

A future classifier should be evaluated against a labelled dataset containing realistic notification descriptions. The evaluation should include common and uncommon merchants, variations in spelling and wording, and examples from every supported category. The resulting dataset would provide the evidence required to determine whether a trained classifier improves upon the existing deterministic mapping.

For this application, the practical input is generally a short merchant label rather than a long natural-language document. The current mapping therefore uses substring matching rather than a multi-stage NLP pipeline. That design works for familiar terms such as Swiggy, Uber or Amazon, but spelling changes, abbreviations and generic merchant names can reduce coverage. A dependable future dataset should record the original notification text alongside a reviewed category so that training examples reflect actual inputs and can be split into development and evaluation sets.

2.2 Naive Bayes for Text Classification

Naive Bayes is a probabilistic classification family based on Bayes' theorem. For a candidate category c and observed feature vector x, the project presents the formulation P(c|x) proportional to P(c) multiplied by the feature likelihoods P(x_i|c). The prior represents how common a category is in the training data, while the conditional probabilities represent the association between observed features and that category.

The Multinomial form is commonly suited to discrete text features such as token counts. During training, the system can count how often tokens occur within each category and use those counts to estimate probabilities. During prediction, the probabilities associated with the observed tokens can be combined to produce a score for each category. The category with the highest score becomes the predicted class.

The simplifying independence assumption does not mean that words are truly independent in natural language. It is an approximation that makes the model computationally simple. For a mobile application, this simplicity can be useful because training and inference can be lightweight. However, the model still depends strongly on the quality of its labelled examples and the coverage of its vocabulary.

In the supplied runtime code, these probability calculations are not currently performed. The class name may suggest Naive Bayes, but the observed implementation is keyword based. This distinction is documented because a report should describe the behaviour that the application actually performs. A future implementation can use the same category structure while replacing the fixed rules with learned probabilities.

A complete Multinomial Naive Bayes implementation would first define a vocabulary, count tokens for each labelled category, estimate category priors and token likelihoods, and then score a new transaction description. Smoothing would be needed so that an unseen token does not force a category score to zero. The current source does not perform these steps; its class name is historical rather than evidence of a trained model. The proposed method is included as a candidate future upgrade and should only be called implemented after the probability calculation is connected to the app and tested on labelled examples.

2.3 Regression and Spending Forecasting

Regression is used when the target value is numeric rather than a discrete class. In the expense-management project, the target is the amount of monthly expenditure and the explanatory variable is the ordered month index. Linear Regression fits a straight-line relationship to historical monthly totals and then evaluates the line at the next month index.

The simplicity of Linear Regression makes it attractive for an educational mobile prototype. The mathematical model is easy to explain, requires little computational overhead and can be implemented directly without a complex external forecasting service. It also makes the relationship between historical observations and the forecast visible to the developer.

However, personal expenditure is not necessarily linear. A student may have seasonal travel costs, a household may have irregular bills, or a user may make a one-time purchase that changes one month dramatically. A linear trend can smooth these variations rather than explain them. The application therefore treats the forecast as an estimate and does not present it as a guaranteed amount.

Forecast evaluation must be chronological. A model should be trained using earlier monthly observations and evaluated on later months so that information from the future does not leak into the past. MAE and RMSE are appropriate descriptive measures for this purpose. The current project material does not report these values, so the expanded report presents them as future evaluation measures rather than achieved results.

The current predictor in LinearRegressionPredictor.kt uses the index of each item in the supplied monthly-total list as the independent variable. It estimates a slope and intercept and evaluates the line at the next index. When there is one total, the code uses that value as a baseline; an empty list returns no prediction. Since the monthly query returns only months represented by saved transactions, gaps are not automatically inserted as zero. This detail matters when interpreting a trend because missing months and months with zero spending are not equivalent.

2.4 Mobile Expense Applications and Workflow Comparison

Mobile expense applications generally provide some combination of manual entry, category organisation, summaries and budget features. Manual entry offers control because the user explicitly supplies the information. Automatic capture can reduce effort but introduces dependencies on notification availability and message formatting. The project combines both approaches to obtain a practical balance between automation and coverage.

From a workflow perspective, the important sequence is capture, validate, classify, store, aggregate and present. If any early stage fails, later calculations can be affected. For example, a wrongly parsed amount can distort the monthly total, and a duplicated transaction can make the forecast appear higher than it should be. The project therefore treats data quality as a cross-cutting concern rather than an isolated database issue.

The local-first design also differs from systems that depend on cloud storage for every operation. Keeping core transaction records on the device can reduce network dependency and simplify the prototype architecture. It does not remove the need for secure device access, permission awareness and responsible handling of backups, but it keeps the main workflow compact.

The project’s main contribution is therefore integration rather than the invention of a new algorithm. It brings notification capture, manual entry, local persistence, category assignment, forecasting and budget guidance into one Android workflow. The value of the design should be judged by functional completeness, usability, maintainability and future evaluability.

Workflow quality is determined by the weakest upstream step. If an event is missed by the notification filter, no parser or classifier can recover it; if the amount is extracted incorrectly, the database and dashboard will preserve and display the wrong value unless it is corrected. For that reason, the application needs a visible transaction history and a usable manual-entry route. These features provide a means to review records rather than treating automatic capture as an infallible background process.

2.5 Local Storage, Privacy and Data Integrity

SQLite is a lightweight relational database suitable for local application data. In this project, transaction records need to survive application restarts and support queries for history and monthly aggregation. A local database is appropriate because the prototype does not require a remote service for its core functions.

Data integrity is especially important in an expense application because every downstream calculation depends on stored values. Merchant text should not be empty when it is required for identification, amounts should be numeric and timestamps should be consistent. Duplicate handling should be applied before a notification event is inserted when the event appears to represent a previously stored transaction.

Privacy is connected to both notification access and local storage. Notification access can expose content belonging to other applications, so the permission should be requested transparently. The application should process only the information necessary for the expense workflow and should not imply that notification access is equivalent to direct access to a bank account.

Future versions can strengthen data integrity through explicit schema design, constraints, indexes, migration rules and retention policies. These details were not fully specified in the source material and should be verified against the actual application before being described as implemented. The expanded report therefore distinguishes confirmed project behaviour from engineering recommendations.

The design is technically compact because the supplied project performs the main data transformations locally. The classifier uses string matching, the predictor uses arithmetic on a list of monthly totals, and the SQLite helper queries local records. These choices reduce deployment dependencies but also limit learning capability and forecasting sophistication.

2.6 Research Gap and Integrated Design Opportunity

The project identifies a practical gap between transaction recording and meaningful personal spending analysis. Manual systems require effort at every transaction. Rigid rule-based systems may categorise common merchants but can struggle with unfamiliar descriptions. A user may also need separate tools to obtain a transaction history, monthly summary, prediction and budget-oriented view.

The proposed application addresses this gap through integration. It does not claim to introduce a novel classification algorithm or a novel forecasting method. Instead, it combines established components into a single mobile workflow. This is appropriate for a mini project because integration itself involves important engineering decisions around permissions, data validation, storage, user interface and failure recovery.

The research gap also includes evaluation. A system may appear intelligent because it produces categories and predictions, but those outputs need quantitative assessment before performance claims are made. The supplied project material explicitly notes the absence of a labelled dataset and measured classification and forecasting metrics.

Future research can therefore focus on building a transaction dataset, comparing deterministic keyword mapping with Naive Bayes, and evaluating alternative forecasting models using chronological validation. Such work would transform the current prototype from an implementation demonstration into an experimentally evaluated system.

The integrated design opportunity is to make data collection and simple analysis available in one small application without requiring a separate backend. Its value should be judged by whether the steps work together reliably and whether the user can understand the generated output. The present implementation is a baseline: fixed merchant rules prioritise transparency, while Linear Regression provides an interpretable numerical estimate. A future experiment can compare these baselines with trained alternatives using the same held-out transactions and chronological monthly data.

2.7 Evaluation Concepts for Classification and Forecasting

Classification evaluation requires a known expected category for each test example. Accuracy measures the fraction of correct predictions, while precision and recall provide category-specific perspectives. F1-score combines precision and recall. A confusion matrix is especially useful because it shows which categories are being confused with one another.

Forecasting evaluation compares a predicted monthly amount with the observed total for that month. MAE provides an interpretable average absolute error, while RMSE gives more weight to larger errors. Both measures should be calculated on data that was not used to fit the model being evaluated.

An important principle is separation between development data and evaluation data. If the same transaction examples are used both to construct a classifier and to report its performance, the resulting metric may overstate generalisation. Similarly, using future monthly totals while building a past forecast would create temporal leakage.

The current report intentionally does not manufacture these measurements. Instead, it documents what would need to be collected and calculated in a future experiment. This preserves the distinction between functional evidence and quantitative evidence.

For category evaluation, each test notification should have a human-verified expected category before the application output is inspected. A confusion matrix can then show, for example, whether groceries are repeatedly confused with shopping. Precision helps assess how often the predicted category is correct, while recall shows how many examples belonging to a category were found. For forecasting, predictions must be recorded before the target month is observed and then compared with that month’s actual total. A shared, reproducible test plan is more informative than selecting a few favourable examples.

2.8 Chapter Summary

Related work establishes that transaction categorisation can be treated as a text-classification problem and that Linear Regression can provide a simple baseline for expenditure forecasting. These methods are understandable and lightweight enough for a mobile prototype. At the same time, both methods have assumptions that must be acknowledged.

The current project implementation uses a deterministic merchant-keyword mapping rather than a trained Naive Bayes classifier. This is a useful baseline because it is transparent, but it does not learn from examples. The forecasting component is similarly simple and should be evaluated against future observed totals before its accuracy can be discussed.

The chapter also highlights the importance of local storage, data integrity, privacy and workflow integration. The project's contribution is the combination of these elements in a single Android application rather than a new mathematical algorithm.

The next chapter converts these design goals into explicit functional and non-functional requirements and examines whether the proposed solution is technically, operationally and economically feasible.

The central lesson from the related concepts is that a simple method is useful when its role and limitations are clear. Keyword categorisation provides a transparent starting point, and Linear Regression offers an understandable trend baseline, but neither removes the need for correct transaction data. Local persistence supports a compact architecture, while permissions and user review remain important for privacy and completeness. The next chapter translates these observations into concrete functional requirements, quality requirements and feasibility considerations for the supplied Android prototype.

CHAPTER 3 – REQUIREMENTS AND FEASIBILITY 3.1 System Requirements Overview

Requirements define the expected behaviour and quality of the application. For this project, the requirements originate from the need to capture expenses, organise them, store them locally and provide useful summaries. The application must support two input paths: supported payment notifications and manual entry. Both paths should converge on a common transaction representation so that later functions do not need to know how the record was created.

A valid transaction should contain enough information to identify the expense and support later aggregation. The source material identifies merchant name, amount, category and timestamp as core fields. The application should validate required values before persistence and should avoid creating a record when parsing produces an uncertain amount or empty merchant field.

The application also requires presentation functions. Users should be able to inspect transaction history, see monthly totals and access the prediction and budget screens. These screens must distinguish actual recorded values from estimated values. This distinction is particularly important because a forecast is derived from historical data and can change when the stored transaction history changes.

Quality requirements include usability, privacy, reliability and maintainability. The prototype should work without unnecessary network dependency, should request notification access explicitly and should provide a manual fallback. The architecture should also keep parsing, categorisation, storage and prediction conceptually separate so that future improvements can be implemented independently.

ID	Requirement	Expected behaviour
FR-1	Notification input	Process supported readable notifications after permission.
FR-2	Manual input	Allow merchant, amount, category, date and time to be entered.
FR-3	Parsing	Extract or accept required transaction fields.
FR-4	Categorisation	Assign a category using the current category component.
FR-5	Persistence	Store transaction data locally in SQLite.
FR-6	Duplicate handling	Check recent merchant-and-amount matches before saving.
FR-7	History	Display transactions and monthly expenditure totals.
FR-8	Forecast and budget	Display a spending estimate and income-informed guidance.

3.2 Functional Requirements

Functional requirements describe what the system must do. The first requirement is notification input: after the user grants notification access, the application should process supported readable payment notifications. The second requirement is extraction: the system should attempt to identify merchant and amount information. The third requirement is manual input: the user should be able to enter an expense when automatic extraction is unavailable.

The fourth requirement is category assignment. The current implementation assigns a category through merchant-keyword matching. The fifth requirement is persistence, which stores the transaction in SQLite. The sixth requirement is duplicate handling, which compares merchant and amount within a recent time window before saving. The seventh requirement is reporting, which provides history and monthly totals.

The eighth requirement is forecasting and budget guidance. When sufficient monthly history exists, the application can estimate next-month expenditure using Linear Regression. The budget screen allows the user to provide monthly income and view category-oriented guidance. These outputs depend on the stored history and should not be interpreted as validated financial advice.

Requirements are best understood as a connected chain. If the parser fails, classification cannot operate; if the stored amount is incorrect, monthly aggregation is affected; if the history is incomplete, the forecast may be misleading. Therefore, functional requirements must be considered together with validation and quality requirements.

The functional requirements should be verified through actions that correspond to the user journey rather than by checking only that a screen opens. For example, a valid notification should be filtered, parsed, categorised, checked for duplicates and stored; the same transaction should then appear in history and affect the appropriate monthly total. A manually entered expense should reach the same storage path. A prediction should read from the saved monthly totals and handle empty history. This end-to-end perspective helps identify defects that would not appear in isolated screen checks.

3.3 Non-Functional Requirements

Usability requires that routine expense entry and review be understandable without advanced financial knowledge. Important information should be visible without forcing the user to interpret technical terminology. Transaction amount, merchant, category and date should be readable, while prediction areas should clearly state that the value is estimated.

Performance should be adequate for local operations. The project does not provide formal benchmarks for response time, memory use or battery consumption, so the report does not claim specific performance figures. Nevertheless, the architecture is designed to keep notification parsing and database operations lightweight. Large remote services are not required for the core workflow.

Reliability means that the application should not treat every notification as a valid expense. Parsing failures, missing fields and duplicate events should be handled safely. Manual entry provides a recovery path. Data integrity also requires consistent numeric amounts and timestamps because errors in these fields propagate into monthly totals and predictions.

Maintainability is supported by modular design. Notification processing, parsing, classification, persistence, aggregation and prediction can be separated conceptually. Privacy requires explicit permission handling and local transaction storage. Compatibility remains dependent on Android behaviour and on the notification formats generated by payment applications.

Aspect	Requirement / design consideration
Privacy	Keep transaction storage on device and request notification access explicitly.
Reliability	Provide manual entry when automatic parsing is unavailable.
Usability	Use clear transaction fields, history, monthly totals and forecast labels.
Data integrity	Validate required fields and reduce likely duplicate records.
Maintainability	Keep parsing, categorisation, prediction and storage separable.

3.4 Intended Users and Use Cases

The primary user is an individual who wants to monitor personal spending. The user grants notification access, performs supported digital payments, reviews captured transactions and enters missing expenses manually. The user can then inspect transaction history, monthly totals, predictions and budget-oriented guidance.

In the automatic-capture use case, a payment notification arrives and the notification listener receives the event. The application examines the readable content and attempts to identify the merchant and amount. If the required fields are available, the record proceeds through duplicate checking and category assignment before being stored. If extraction fails, the user can use manual entry.

In the manual-entry use case, the user supplies transaction information through the Add Expense interface. The application validates the required values, assigns or records the category according to the available workflow and stores the record. Because both paths produce the same core transaction representation, later screens can display them together.

Other use cases include reviewing history, examining monthly spending and entering income for the budget recommendation screen. The system should keep these use cases independent of the original input source. A transaction captured from a notification and a transaction entered manually should both be available to the same reporting and aggregation functions.

Use case	Actor	Main outcome
Grant notification access	User	Automatic input path becomes available.
Capture transaction	System	Supported notification is processed.
Add expense	User	Manual transaction is stored.
Review history	User	Saved transactions are displayed.
View monthly spending	User	Transactions are aggregated by month.
View forecast	User	Estimated next-month amount is shown when sufficient history exists.
Set income and view budget	User	Income-informed category guidance is displayed.

3.5 Technical Feasibility

Technical feasibility is supported by the use of established Android technologies. Kotlin provides the application logic, Jetpack Compose provides the user interface, SQLite provides local persistence and NotificationListenerService provides access to notification events after user authorisation. These technologies can operate within a native Android application and are suitable for a compact prototype.

The classification and forecasting methods are also technically feasible. Merchant-keyword mapping requires only string processing and does not require a machine-learning runtime. The proposed Naive Bayes classifier could be implemented using token counts and probability estimates from labelled data. Linear Regression requires a small set of historical monthly totals and can be calculated with basic arithmetic.

The main technical uncertainty is notification compatibility. The application depends on the text exposed by other applications and on Android permission behaviour. Different notification formats can require different parsing rules. This uncertainty is not a reason to reject the design; instead, it reinforces the need for manual entry and explicit compatibility limitations.

The source material does not specify a complete hardware configuration, minimum SDK, dependency versions or build settings. Those details should be documented from the actual development environment when required. The expanded report avoids inventing such values.

The local database currently stores merchant, amount, category and created_at. The amount is represented as a numeric REAL field for aggregation, while created_at stores a millisecond timestamp. Date and time strings are formatted when records are read for the user interface rather than being the primary stored time values.

3.6 Operational Feasibility

Operational feasibility asks whether the proposed workflow can be used effectively in practice. The most important user interaction is notification access. The user must understand why access is requested and must intentionally grant it through Android settings. The application cannot assume that permission is available automatically.

Once permission is granted, the notification workflow should be as unobtrusive as possible. A supported payment notification can be processed in the background, while the user retains control through the transaction history. When a notification cannot be parsed, manual entry prevents the system from becoming unusable.

The application also needs understandable feedback. A user should be able to recognise whether an expense was stored, whether a duplicate was ignored and whether a prediction is available. The current project material demonstrates these concepts through dashboard, history, monthly spending and budget screens.

Operational success is therefore not only about technical execution. It depends on the user’s understanding of permissions, the clarity of the interface and the ability to recover from unsupported notifications. The dual input design is an important operational safeguard.

The operational workflow is simplest when permission setup is explained once and then the user can return to the main dashboard. The current project stores the entered name in SharedPreferences so the name setup does not need to be repeated after a successful save. Notification access remains separately controlled by Android and may be revoked by the user at any time. A practical guide should explain that disabling this access stops supported automatic capture but does not prevent manual expense entry or review of previously stored data.

3.7 Economic Feasibility

Economic feasibility considers whether the proposed system can be developed and operated with reasonable resource requirements. The supplied project material does not provide a formal cost analysis or deployment budget, so no numerical economic claim is made in this report.

The architecture nevertheless has characteristics that can reduce infrastructure requirements. Core transaction storage is local, and the main categorisation and forecasting operations can be performed on the device. A remote database or continuous server connection is not required for the fundamental workflow described by the prototype.

Development cost also depends on the availability of Android development tools, testing devices and time required to support different notification formats. Additional work would be required if the application were expanded to a large number of payment providers or if a cloud-based synchronisation service were introduced.

For an academic prototype, the local architecture is economically practical because it focuses on the core demonstration without requiring a large backend system. Any production deployment should perform a separate cost and maintenance assessment based on actual requirements.

The largest ongoing maintenance effort is likely to be compatibility rather than server operations. If a payment application changes its notification wording, the filter or parser may need a corresponding update. The static merchant map also needs maintenance when new merchants and categories are introduced. These are realistic costs in development time rather than a quantified financial estimate. Because the supplied project does not include measured deployment costs, paid services or a production budget, the feasibility conclusion remains qualitative and limited to the academic prototype.

3.8 Assumptions, Dependencies and Constraints

The design assumes that supported payment notifications contain enough readable information to identify a merchant and amount. It also assumes that the user grants notification access and that the Android environment permits the relevant notification events to be processed. These assumptions are outside the direct control of the application.

Dependencies include Android notification behaviour, notification text formats, the local database and the availability of sufficient historical monthly data. If notification content is redacted or unavailable, the automatic path cannot guarantee a transaction record. If the transaction history is incomplete, monthly totals and forecasts may also be incomplete.

Duplicate detection is another constraint. Comparing merchant and amount within a recent time window can reduce repeated notification records, but two genuine purchases can have the same merchant and amount within the same period. The mechanism is therefore a heuristic rather than a proof of transaction identity.

The forecasting component is constrained by history length and quality. A small number of months may not represent future behaviour. These constraints should be visible in the documentation and considered in future evaluation rather than hidden behind a single numerical prediction.

The assumptions can be turned into explicit acceptance conditions. A notification should be captured only when the content is readable and matches the supported amount and payment patterns; otherwise it should not create a partly populated expense. A duplicate match should be logged or otherwise made reviewable when practical, because the current merchant-and-amount comparison over two minutes is not a unique transaction identifier. Forecasting should also expose the amount of history used so that the user can judge whether the estimate is based on enough observations.

CHAPTER 4 – SYSTEM ANALYSIS AND DESIGN

4.1 Overall System Architecture

The system architecture is organised around input, processing, persistence and presentation. The input layer contains supported payment-notification capture and manual expense entry. The processing layer filters notification content, parses merchant and amount fields, validates values, checks likely duplicates and assigns a merchant-keyword category. The persistence layer stores expense rows in SQLite.

The two input paths converge on a shared expense record. A supported notification is filtered and parsed before it is saved; a manually entered expense supplies the merchant and amount directly. Both paths then use the same local storage and can appear in history and monthly summaries. If a notification is unavailable or cannot be parsed, manual entry remains the fallback.

The current category component performs deterministic keyword matching. Although the source file is named NaiveBayesClassifier.kt, it does not train a probabilistic model or calculate Bayes probabilities. The architecture keeps this component isolated so a trained classifier could replace it in future work without redesigning the main data path.

Monthly totals are retrieved from saved expense records and passed to LinearRegressionPredictor for a basic next-month estimate. The prediction depends on the completeness and correctness of the transaction history. Income-based budget recommendations are planned scope only: no corresponding screen, saved income field or recommendation formula was found in the supplied project ZIP.

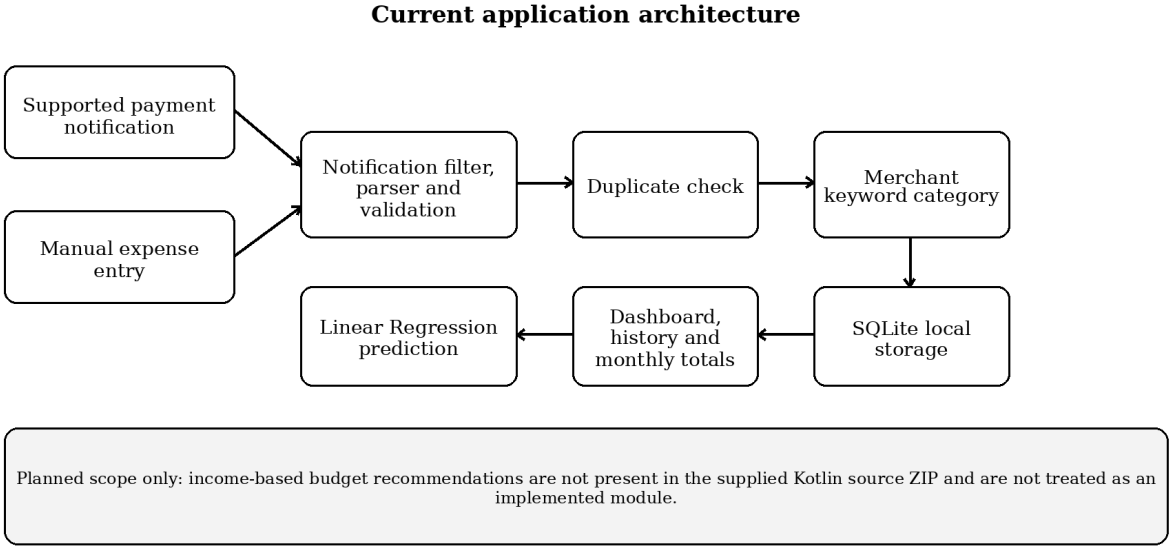


Figure 1. Current system architecture; budget recommendations are shown as planned scope rather than a current module.

4.2 Module Design and Responsibilities

The Notification Listener module receives supported Android notification events after the user grants access. Its responsibility is to forward relevant readable content for processing rather than to assume that every notification is an expense. This separation prevents the listener from becoming responsible for business logic that belongs to the parser.

The Parsing and Validation module examines text and attempts to identify merchant and amount. It should reject blank text, invalid numeric amounts and missing merchant information. The Manual Entry module supplies a valid transaction record when automatic extraction cannot provide one.

The Categorisation module receives merchant or transaction text and produces a category. The current implementation searches predefined keywords and chooses the longest matching keyword. The Database module stores and retrieves records, while the Prediction module aggregates monthly totals and estimates future spending.

The presentation modules consume stored data and display it to the user. The dashboard provides a summary, the history screen provides transaction-level detail, the monthly spending screen provides aggregation, and the budget screen combines income with spending information. Clear module boundaries make testing and future modification easier.

4.3 End-to-End Data Flow

A transaction begins at either the notification listener or the manual-entry form. In the notification path, the system receives readable text and passes it to the parser. In the manual path, the user directly supplies the required values. Both paths should produce a merchant and amount before the transaction proceeds.

The next stage is validation and duplicate checking. The amount must be a valid numeric value and the merchant should be meaningful. The duplicate mechanism compares merchant and amount within a recent time window. If a likely duplicate is found, the event can be ignored rather than stored again.

After validation, the transaction is categorised and inserted into SQLite. The dashboard and history screens can then retrieve the stored record. Monthly totals are calculated from the same records, which keeps reporting consistent with the transaction history.

For forecasting, the system groups transaction amounts by calendar month and creates an ordered series. Linear Regression uses the month index as x and the monthly total as y. The fitted relationship is evaluated at the next month index to produce an estimate. The value is displayed separately from actual spending.

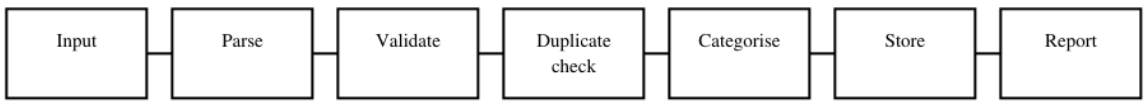


Figure 2. Notification-to-storage transaction lifecycle.

4.4 Local Database Design

SQLite is used as the local persistence layer. The core transaction fields identified by the project are merchant name, amount, category and timestamp, with an identifier used to distinguish records. These fields support the history screen, monthly grouping and category-based summaries.

A conceptual record can be represented as Transaction ID, Merchant Name, Amount, Category, Date and Time. The exact physical schema should be confirmed against the implementation. The source material does not provide complete migration rules, indexes or retention policies, so those details are treated as engineering considerations rather than completed features.

Database operations should preserve numeric amounts consistently. Storing amounts in a form that can be reliably converted to numbers is important because the monthly aggregation sums these values. Timestamps should also be stored consistently so that transactions can be ordered and grouped by month.

Local persistence provides an important reliability benefit: the transaction history remains available after the application is closed and reopened, subject to the application data remaining intact. It also keeps the core workflow independent of a remote server.

Monthly aggregation is performed with SQLite strftime over the created_at timestamp, grouping records by calendar month and ordering months chronologically. The method returns a list of totals; it does not itself retain missing zero-spend months, so a forecasting sequence can represent only months that appear in the grouped query.

4.5 Data Dictionary and Integrity Rules

Merchant name identifies the business or transaction source. It supports history display, duplicate checking and categorisation. Amount represents the expense value and is essential for monthly totals and forecasting. Category stores the label assigned by the current classification workflow. Timestamp identifies when the transaction occurred or was registered.

Integrity rules should prevent an empty merchant or invalid amount from entering the database. When a notification contains several numbers, the parser should select the amount according to the supported message pattern and should reject the message when the result is ambiguous. Manual entry should apply equivalent validation rules.

Duplicate checking is not a database uniqueness constraint because repeated legitimate purchases can share merchant and amount. Instead, the project uses a recent-time-window comparison. This is a practical compromise for notification events, which can be delivered more than once.

The data dictionary should remain aligned with the actual implementation. The report deliberately avoids adding unsupported fields such as bank account numbers, receipt identifiers or cloud synchronisation flags. Additional fields can be introduced in future versions after the application design has been updated.

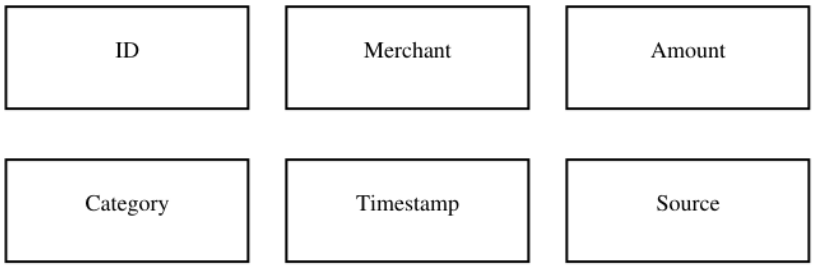


Figure 3. Conceptual local expense record and transaction fields.

4.6 Dashboard and User Interface Design

The current Compose interface presents the main functions of the expense tracker through the Home, Transactions, Add Expense, AI and Prediction routes defined in MainActivity.kt. The dashboard summarises locally stored expenses; the transaction screen presents saved records; and the Add Expense form provides a manual alternative when automatic capture is unavailable.

The AI screen provides a way to test the merchant-category mapping. In the supplied source, this is a deterministic keyword lookup rather than a trained probabilistic classifier. The Prediction screen uses saved monthly totals to display an estimated next-month amount when the predictor returns a value. The estimate should be interpreted as a simple trend baseline, not a guarantee.

The screen layout and reporting should make it possible to distinguish recorded transactions from calculated estimates. Empty history, invalid manual input and unsupported notifications need understandable fallback states. The screenshot appendix provides interface references from the supplied report, but only screens represented in the current source are considered part of the current navigation.

The earlier Budget Recommendation screenshot is retained as a design reference only. It does not correspond to a current route in MainActivity.kt, and the uploaded source does not implement an income-entry field or category recommendation calculation.

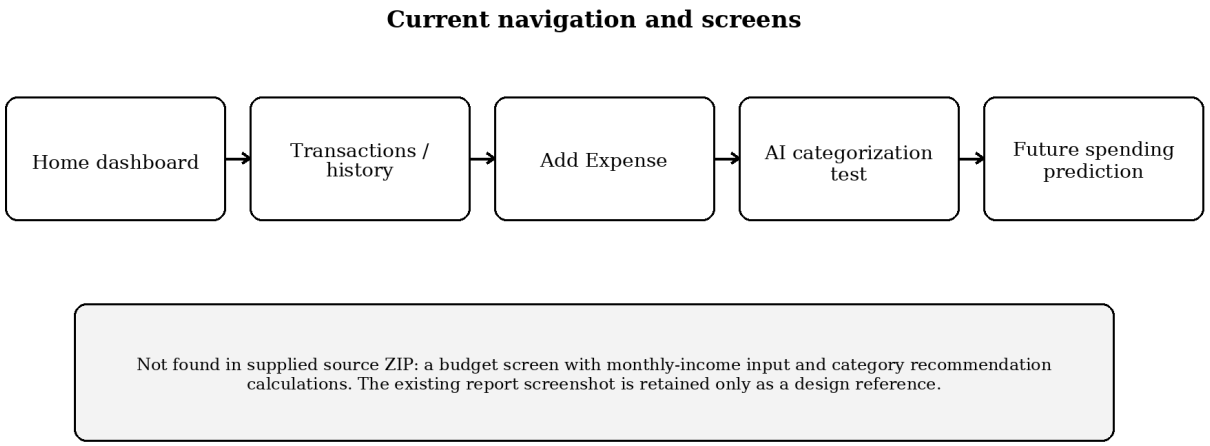


Figure 4. Current navigation and screen functions based on the supplied MainActivity.kt.

4.7 Validation, Error Handling and Recovery

Validation is required at every input boundary. Notification text should be checked for usable content, the extracted amount should be numeric and the merchant should not be empty. Manual entry should apply similar checks before saving. Validation prevents low-quality data from propagating into later calculations.

Parsing errors should not create invented transactions. When the notification does not match a supported pattern, the safest response is to ignore automatic insertion and allow manual entry. This design acknowledges that notification formats can vary and that false transaction records can be more harmful than missed automatic capture.

Duplicate handling is another recovery mechanism. The supplied implementation checks for a recent record with the same merchant and amount. This can reduce repeated records caused by duplicate notification events. However, because identical legitimate purchases are possible, the mechanism should be described as heuristic.

Database failures should be handled without silently losing the user’s input. The user should be able to review history after saving an expense. Future versions can add more explicit error messages and recovery logs, but such features should be implemented and tested before being claimed.

The current manual-entry checks are more direct than notification parsing because the user supplies the merchant and amount separately. The form rejects an empty merchant and any amount that is absent, non-numeric or not greater than zero before saving. For notification input, the parser may return null when it cannot identify both required fields, and the listener then stops that event rather than inserting an incomplete record. These safeguards reduce basic data-quality failures, but they do not validate the real-world correctness of the entered or extracted amount; reviewing transaction history remains important.

The recovery sequence can be described as a set of gates. For a notification, the listener first checks for payment-related text; the parser then looks for a supported currency prefix and a merchant phrase beginning with "to" or "at". If parsing returns no transaction, the event is not inserted. A candidate that passes parsing is checked against recent records before saving. Manual entry uses separate fields and rejects a blank merchant or an absent, non-numeric or non-positive amount. These checks prevent common malformed records, but they do not verify a transaction against a bank. A notification can still be misread when it contains several currency amounts or unfamiliar merchant wording, so the history screen remains important for user review. The two-minute duplicate rule is a heuristic, not proof of transaction identity.

4.8 Budget Recommendation Design (Planned Feature)

The project title includes budget recommendations, and an earlier report screenshot depicts an income field and category-oriented guidance. That screenshot and design are retained as the intended direction of the project, but they do not match the current source ZIP.

The supplied Kotlin files implement a home dashboard, transaction history, manual expense entry, an AI categorisation test screen and a Linear Regression prediction screen. They do not contain a monthly-income input state, a saved income field, a BudgetRecommendationScreen function or a category allocation formula. The current database schema stores expense records only.

A future implementation should define the recommendation method before presenting it as a feature. For example, it could allow the user to enter monthly income, store the value locally, calculate recorded spending by category, and compare actual amounts with user-defined category limits. Any such method should explain its assumptions and avoid presenting suggestions as personalised financial advice.

Status against the uploaded source: this subsection describes the intended budget-recommendation design, not an implemented module. The current ZIP contains no monthly-income input state, income persistence operation, category allocation calculation or BudgetRecommendationScreen function. The screenshot later in Appendix A is therefore labelled as a design reference rather than runtime evidence.

A practical future design should make its calculation visible to the user. The form could accept monthly income and an optional savings target, validate the values and save them locally. A simple first step would calculate available spending as income minus the chosen savings target, then compare that amount with actual spending from the stored transaction history. Category ceilings should be editable or based on clearly disclosed rules instead of being presented as universally correct percentages. The interface should explain when income is missing or transaction history is incomplete rather than generate an unsupported recommendation. Tests should cover blank income, zero or negative values, very high values and months with no expense records. These are proposed requirements only: the supplied ZIP contains no current income-entry screen, persisted income field or budget-recommendation calculation.

4.9 Security and Privacy Design

Security considerations begin with notification access. The permission can expose notification content from other applications, so it should be requested deliberately and explained clearly. The application should process only the information needed for expense tracking and should not treat notification access as unrestricted access to a user’s financial accounts.

Local SQLite storage reduces the need to transmit transaction records to a remote service. However, local storage still depends on device security. A future production version could consider encryption, protected backups and explicit data-retention controls. These features are not claimed as part of the current prototype.

Privacy also affects the user interface. Sensitive transaction details should not be displayed unnecessarily in contexts where other people can easily view the screen. Error messages should avoid exposing raw notification text when it is not needed.

The design principle is data minimisation: capture the information needed to perform the expense workflow, retain it for useful application functions and avoid adding unsupported external data sources. This principle supports both user trust and maintainable architecture.

The local-first design reduces the need to transfer transaction records to a server for ordinary use, but it does not automatically guarantee confidentiality against someone who can unlock the device or inspect its app data. The reviewed database helper uses SQLite and the project includes Android backup-rule resources; the supplied files do not show a custom encryption layer. A production-oriented revision should review platform backup settings, minimise retained notification text, document deletion behaviour and consider an appropriate encryption strategy if the threat model requires it. Permission wording should also explain that notification content can be sensitive.

A source-level privacy concern is that the notification listener writes debugging messages containing payment-detection details and parsed merchant and amount values. Such logging is useful while diagnosing input formats, but it can expose spending information in device logs; a release build should remove, redact or guard these messages behind a debug-only configuration. The database is local SQLite, and the submitted source does not show a database-encryption layer or remote synchronisation service. The report therefore describes the data as locally stored, not as encrypted or guaranteed confidential. Data minimisation also means ignoring unrelated notification fields and not retaining the full notification body when the parsed merchant and amount are sufficient. Users should be told why notification access is needed and how to revoke it through Android settings.

CHAPTER 5 – METHODOLOGY AND ALGORITHMS

5.1 Processing Methodology Overview

The methodology follows a sequence from transaction acquisition to user-facing analysis. First, the application obtains information from a supported notification or manual input. Second, the information is parsed or validated. Third, duplicate checking is performed. Fourth, a category is assigned. Fifth, the transaction is persisted in SQLite. Finally, stored records are aggregated for reporting, forecasting and budget guidance.

The methodology is designed so that the same transaction representation can be produced by different input paths. This avoids maintaining separate data-processing logic for notification transactions and manual expenses. It also means that a transaction entered manually can participate in the same monthly summaries and forecasts as an automatically captured transaction.

The classification and forecasting stages have different mathematical goals. Categorisation selects a discrete label, while forecasting estimates a continuous numeric value. The project describes Multinomial Naive Bayes for the former and Linear Regression for the latter, while the current runtime classifier uses deterministic keyword mapping.

Formal evaluation is intentionally separated from implementation. Functional testing can determine whether screens and workflows operate as intended. Machine-learning evaluation requires labelled category data and observed future monthly totals. The methodology therefore includes both a working prototype path and a future experimental path.

The pipeline has one shared transaction representation, which limits the number of places that must understand the source of a payment. The notification service handles platform events and filters them; the parser handles text-to-fields conversion; the category mapper assigns a label; the database helper stores and queries records; and the dashboard reads the resulting data. Monthly aggregation and prediction operate on the stored values rather than directly on raw notifications. This separation makes failures easier to isolate and helps future changes remain local to the relevant component.

5.2 Notification Capture and Permission Process

Android NotificationListenerService is the mechanism identified by the project for receiving notification events. The user must grant notification access through Android settings. The application should explain the purpose of this permission because notifications can contain private information from many applications.

When an event is received, the application examines the available readable content. The listener itself should not decide whether the event is a valid expense. It forwards the text to the parsing stage, where the message is interpreted according to the supported patterns.

Notification capture is inherently dependent on external applications. A payment provider can change its notification wording, hide content or stop displaying notifications. Android behaviour can also affect which events are available. The application therefore cannot guarantee complete coverage.

Manual entry provides the required fallback. If the notification cannot be processed, the user can enter the expense directly. This is important for cash transactions, unsupported providers and messages that contain insufficient information.

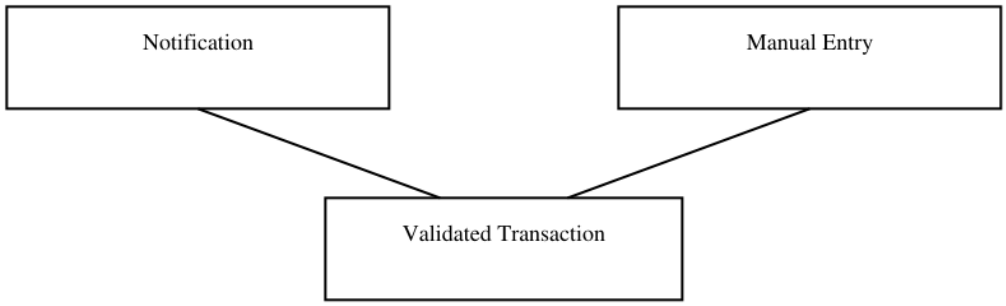


Figure 5. Supported notification and manual-entry paths.

5.3 Transaction Text Parsing and Validation

The parser attempts to identify two key fields: merchant and amount. The amount may appear with currency symbols or prefixes, while the merchant may appear after words such as “to” or “at”. The supplied code uses regular expressions to recognise supported patterns and converts the extracted numeric text into a Double.

Parsing must distinguish the transaction amount from other numbers such as balances or reference identifiers. This is why pattern design is important. A parser that simply selects the first number in a notification can produce incorrect records. The implementation instead looks for an amount associated with a supported currency pattern.

Merchant extraction also requires validation. The extracted merchant should not be blank and should contain meaningful text. When a supported pattern cannot be found, the parser returns no transaction rather than inventing a value. This conservative approach protects the database from uncertain records.

Future versions can improve parsing by supporting more notification templates, additional currency formats, punctuation variations and multilingual messages. Such extensions should be accompanied by test cases so that improvements do not break previously supported patterns.

The amount expression accepts rupee-symbol, Rs/Rs. and INR prefixes, optional commas and up to two decimal places. Merchant extraction is narrower: it searches for a phrase beginning with “to” or “at” followed by up to four words made from the supported character set. Other notification templates can be rejected even when they describe a valid payment.

5.4 Duplicate Transaction Detection

Notification systems may deliver the same event more than once. If every event were inserted independently, the transaction history and monthly totals could be inflated. The project therefore checks whether a recent record exists with the same merchant and amount.

The supplied implementation uses a recent two-minute window in its duplicate-check function. This is a practical heuristic for repeated notification delivery. It does not establish that two records represent the same real-world transaction because two legitimate purchases can occur at the same merchant for the same amount within that window.

More robust duplicate detection could use notification identifiers, transaction references, timestamps and user confirmation. A future design might also store a source field describing whether the transaction came from a notification or manual entry. These additions should be evaluated before being treated as part of the current system.

Duplicate handling should be applied before the transaction affects monthly aggregation. This illustrates the dependency between data quality and analytics: a small input-level error can become a larger reporting error when many transactions are aggregated.

The duplicate test uses an exact merchant string, exact amount and a two-minute look-back window. This is a practical heuristic for duplicate notification events, including cases where a payment appears through more than one notification channel. It can also collide with two genuine purchases of the same amount at the same merchant within that short window.

5.5 Multinomial Naive Bayes Formulation

The project presents Multinomial Naive Bayes as the intended probabilistic model for transaction categorisation. For category c and feature vector x, the posterior probability can be represented proportionally as P(c|x) proportional to P(c) multiplied by the likelihoods of the observed features given c. The model uses labelled examples to estimate the required probabilities.

In a transaction classifier, features can be tokens extracted from merchant descriptions. During training, the system counts token occurrences within each category. These counts are converted into probability estimates. During prediction, the observed tokens contribute to the score of each category and the category with the largest score is selected.

Probability smoothing is normally important because a token that never appeared in a category should not necessarily make the entire probability zero. A production implementation would also need tokenisation, case normalisation, vocabulary management and a defined category set.

The current runtime implementation does not perform these probability calculations. It uses deterministic merchant-keyword mapping instead. Therefore, the equation in this chapter documents the intended theoretical approach and not a measured runtime calculation.

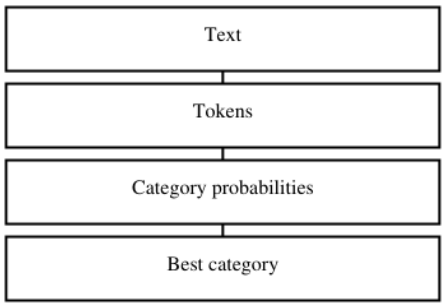


Figure 6. Proposed Multinomial Naive Bayes classification workflow.

5.6 Current Merchant-Keyword Categorisation

The supplied classifier normalises the merchant name to lowercase and checks configured keywords. Examples in the source include food-related merchants such as Swiggy and Zomato, travel-related names such as Uber, Ola and Rapido, and shopping-related names such as Amazon, Flipkart and Myntra.

The implementation chooses the longest matching keyword. This rule is useful when one merchant name can contain multiple configured strings. If no keyword matches, the classifier returns “Other”. The behaviour is deterministic, which means the same merchant text produces the same category when the mapping remains unchanged.

The advantage of this method is transparency. A developer can inspect the mapping and immediately understand why a merchant received a category. The disadvantage is limited generalisation. A new merchant will not be recognised until its keyword is added or another classification mechanism is introduced.

The report therefore avoids describing the current mapping as a trained machine-learning classifier. Future work can compare this baseline against Multinomial Naive Bayes using a labelled dataset. Such a comparison would show whether learned text features improve coverage and category-level performance.

The actual category map includes Food, Travel, Bills, Shopping, Entertainment, Groceries and Healthcare, with Other used when no keyword matches. Matching is case-insensitive and the longest matching keyword wins. This is deterministic rule-based classification; it does not update probabilities from user feedback or labelled training examples.

Because the rule uses substring containment, it does not understand the full meaning of a transaction description. A configured fragment may match even when the surrounding wording is ambiguous, while a valid merchant with a new spelling or alias may fall through to "Other". The longest-keyword rule helps when a longer configured phrase is more specific than a shorter one, but it cannot judge whether a notification represents an actual purchase. These limitations are predictable and make the current mapping a useful baseline for repeatable tests. A future trained classifier should be compared against this same baseline using held-out examples and category-wise error analysis. It should not be described as an improvement until those comparisons have been measured.

5.7 Linear Regression Formulation

The spending forecast uses the standard Linear Regression form y = β0 + β1x + ε. Here, x is the ordered month index and y represents monthly expenditure. β0 is the intercept, β1 is the slope and ε represents the residual error.

The implementation first creates an ordered list of monthly values. It then calculates sums of x, y, x multiplied by y and x squared. The slope is calculated from these sums, followed by the intercept. The next month index is substituted into the fitted equation to obtain the forecast.

A minimum of two monthly values is required by the supplied predictor before a regression line can be calculated. If the denominator becomes zero, the implementation falls back to the last value and ensures that the returned estimate is not negative. This behaviour is part of the supplied code and is documented here rather than replaced with a theoretical alternative.

The model is deliberately simple. It is appropriate as a baseline and is easy to interpret, but it cannot automatically model seasonality or sudden changes. The forecast should therefore be treated as an estimate derived from the current history.

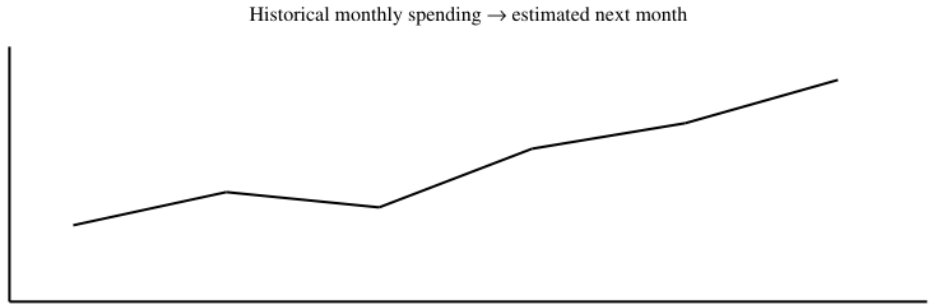


Figure 7. Monthly expenditure forecasting workflow.

5.8 Monthly Expenditure Aggregation

Forecasting depends on a correctly constructed monthly sequence. Each transaction has an amount and timestamp. The aggregation process groups transactions by calendar month and sums the amounts belonging to each group. The result is a time-ordered list suitable for regression.

Aggregation is sensitive to missing and duplicate records. If an expense is omitted, the monthly total becomes too low. If a notification is stored twice, the total becomes too high. These errors can affect both the displayed monthly summary and the next-month prediction.

The month index represents the position of each month in the available history. The predictor uses these indexes as x values and the monthly totals as y values. The next index is then used for the forecast. This approach treats the sequence as equally spaced observations.

Future forecasting experiments can compare this baseline with other methods, but any alternative should be evaluated chronologically. The purpose of the current implementation is to demonstrate a transparent end-to-end forecasting workflow rather than to claim state-of-the-art prediction.

The SQL query groups all stored expense rows by local calendar month and sums amount. Ordering by month ascending gives the predictor an older-to-newer sequence. The query does not synthesise absent months with a zero total, so users should interpret the forecast in the context of the transactions actually stored.

One limitation of the current query is that it returns only months containing at least one stored expense; it does not insert a zero for an intervening month with no rows. The predictor later treats the returned totals as consecutive positions in a sequence, so a gap in the calendar can make the trend look more continuous than the source data really is. A stronger implementation would build a complete month sequence from the first recorded month to the last and explicitly decide whether a missing month means zero spending or missing data. That decision should be documented and tested before a forecast is interpreted as a time-based trend. The totals are also affected directly by missing records, duplicate records and any manual deletions.

5.9 Interpretation and Responsible Display of Predictions

A prediction is a model-generated estimate rather than a promise. The value depends on the stored history and on the assumptions of the regression model. If the user’s future spending changes substantially, the estimate may differ from actual expenditure.

The interface should identify the forecast period and separate the estimate from recorded monthly totals. This reduces the chance that a user interprets a model output as an observed fact. If insufficient history is available, the application should avoid displaying a misleading prediction.

The source material explicitly notes that the forecast should be treated as an estimate. The expanded report preserves this limitation. It does not add confidence intervals, accuracy percentages or financial advice because those outputs are not present in the supplied evidence.

Responsible presentation also means explaining the data dependency. Users should understand that missing transactions, duplicate records and incorrect amounts can change the forecast. A transparent interface can encourage users to review history before relying on the summary.

The prediction method returns null for an empty history, uses the only observed total as a baseline when exactly one month is present, and computes a straight-line fit when there are at least two values. The result is rounded to a whole-number amount and clamped so it cannot be negative. These guards make the UI more robust but do not prove forecasting accuracy.

The implementation has three distinct paths. With no monthly totals, the predictor returns null so the screen can communicate that an estimate is unavailable. With exactly one month, it uses that month as a baseline rather than fitting a line. With two or more observations, it calculates a least-squares slope and intercept using sequential month indexes, then evaluates the fitted line at the next index. The predicted value is clamped to zero or greater and rounded to a whole-number amount. The screen also uses a recent portion of the historical series alongside the prediction for its visual summary. These choices make the result straightforward to display, but the algorithm does not calculate a confidence interval, quantify uncertainty or guarantee that future spending will follow the fitted trend.

5.10 Algorithm Limitations and Future Improvements

Naive Bayes depends on representative labelled examples and a simplifying independence assumption. It can struggle with ambiguous merchant descriptions, new merchants and category imbalance. The current keyword approach has an even simpler limitation: it cannot learn new patterns without changing the configured mapping.

Linear Regression assumes that a straight-line trend is useful for the available history. It may not capture seasonal spending, irregular bills or sudden purchases. A small number of observations also makes the fitted trend sensitive to individual months.

Future work should therefore focus on evaluation before optimisation. A labelled transaction dataset can be used to compare keyword mapping with Naive Bayes. Chronological monthly data can be used to measure forecast error. These experiments would provide evidence for choosing more complex methods.

The current algorithms are appropriate as a transparent baseline for the prototype. Their limitations are not failures of the project; they define the next stage of development and show where additional data, testing and modelling work would be most valuable.

Important limitations follow directly from the implementation. Keyword coverage depends on the merchant map; notification formats vary; the parser may miss messages without a “to” or “at” merchant phrase; duplicate detection is heuristic; and linear extrapolation cannot model sudden one-off expenses or seasonal patterns.

CHAPTER 6 – IMPLEMENTATION

6.1 Development Environment and Technology Stack

The application is implemented as a native Android application using Kotlin. Jetpack Compose is used for the user interface, SQLite provides local transaction persistence and NotificationListenerService provides access to supported notification events after the user grants permission.

Kotlin coordinates the main application operations. It provides the logic required to validate user input, process notification text, assign categories, access the database and calculate predictions. The project material does not specify a complete build configuration, dependency list or minimum Android SDK, so this report documents only the technologies that are confirmed by the source.

Jetpack Compose provides a declarative approach to screen construction. This is suitable for an application where the displayed transaction list, monthly totals and budget information depend on changing local state. SQLite provides a simple relational structure for storing the records required by the application.

The technology stack is intentionally compact. It avoids requiring a cloud backend for the main workflow and keeps the classification and forecasting operations local. This supports the project’s privacy-oriented and educational objectives.

The key files for the report are Kotlin source files in the app package. The submitted ZIP does not provide a labelled transaction corpus, model-training routine, evaluation script, or recorded MAE/RMSE/accuracy output. Such assets would be required before claiming experimentally validated AI performance.

6.2 Kotlin Application Logic and Module Integration

Kotlin is responsible for connecting the application modules. Notification events are passed to parsing logic, validated records are checked for duplicates, categories are assigned and valid records are inserted into SQLite. The presentation layer then retrieves information from storage for display.

Separating these responsibilities improves maintainability. A parsing change can be tested without rewriting the database logic. Similarly, the current keyword classifier can be replaced by a trained model without changing the transaction-history screen as long as the classifier continues to return a category.

The Linear Regression predictor is also isolated as a logical component. It receives historical monthly values and returns an estimated next value. The predictor checks whether enough values exist and handles the zero-denominator case described in the supplied code.

The implementation demonstrates how relatively small Kotlin components can be combined into an end-to-end application. The important engineering principle is not the size of any individual class but the clarity of the data flow between components.

NotificationReaderService.kt includes the payment filter, combines the notification title and text fields, calls TransactionParser.parse, assigns a category using NaiveBayesClassifier.classify, checks for a recent duplicate, and saves the expense. After saving, it broadcasts a refresh signal so MainActivity reloads local data.

6.3 User Interface with Jetpack Compose

The interface is built using Jetpack Compose and the supplied MainActivity.kt defines the current screen routing. The navigation enum contains HOME, TRANSACTIONS, ADD_EXPENSE, AI and PREDICTION. These routes correspond to the home dashboard, transaction history, manual expense form, merchant-category test and future spending prediction.

The dashboard presents saved expense totals and summary information. The transaction screen shows saved rows, while Add Expense validates a merchant and amount before writing a record. The AI screen lets the user test a merchant name and observe the keyword-based category result. The prediction screen displays a next-month estimate when the predictor returns a value.

The earlier budget screenshot is not part of this current screen set. It is retained in Appendix A as a design reference only; the supplied code does not contain the corresponding income input and recommendation workflow.

The current Compose interface includes the name setup screen, home dashboard, transaction list, Add Expense form, AI categorisation test screen and prediction screen. This is the screen set verified against MainActivity.kt. An income-based budget screen is not present in this source snapshot.

6.4 SQLite Local Persistence

SQLite stores the transaction records required by the application. The core fields are merchant, amount, category and timestamp, with an identifier supporting record management. These values are sufficient for transaction history and monthly aggregation.

Database persistence means that the application can retrieve records after a normal application restart, provided that application data remains intact. It also gives the prediction module a stable source of historical values rather than requiring the user to re-enter earlier months.

Database correctness depends on input validation. An invalid amount can corrupt monthly totals, while an inconsistent timestamp can place a transaction in the wrong month. Duplicate records can similarly inflate totals. For this reason, persistence should occur only after the processing stages have validated the record.

The source material does not document migration strategies, indexes or backup policies. These are appropriate areas for future engineering work, especially if the prototype is developed into a production application.

DatabaseHelper.addExpense inserts merchant, amount, category and the current timestamp into the expenses table. getAllExpenses reads rows newest-first and formats the timestamp into a date and time for display. deleteExpense removes records by merchant, amount and category, which may match more than one row if duplicates exist.

6.5 Notification Access and User Consent

NotificationListenerService requires user authorisation. The application cannot silently obtain this access, and the user should be informed that notifications may contain private information. The permission is therefore both a technical dependency and a privacy consideration.

After access is granted, the application can receive notification events within the limits imposed by Android and the source applications. The listener passes readable content to the parser. If the notification is redacted or uses an unsupported format, the automatic workflow may not produce a record.

Permission handling should also consider the disabled-permission state. The application should remain usable when notification access is unavailable because manual entry is an explicit part of the project design.

Future versions can improve the permission screen by explaining the purpose of access, showing the current permission state and guiding the user to settings. These improvements would strengthen usability without changing the core architecture.

Notification access is controlled by Android settings rather than enabled silently by the app. MainActivity checks the enabled_notification_listeners setting to determine whether its package is authorised. Compatibility remains dependent on each notification producer exposing readable title, text, bigText or subText fields.

6.6 Manual Expense Entry

Manual expense entry is the primary fallback when automatic capture fails. It also supports cash purchases and transactions from providers whose notifications are not supported. The user supplies the required transaction information and submits it through the interface.

The manual path should apply the same validation and storage rules as automatic transactions. This keeps the transaction history consistent. An expense entered manually should be available to the same monthly aggregation, prediction and budget functions as an automatically captured expense.

The exact fields supported by the current application should be aligned with the actual UI. The project material identifies merchant, amount, category and date/time as core transaction information. The report avoids adding unsupported fields.

Manual entry also provides a practical recovery mechanism. Instead of requiring the parser to recognise every possible notification pattern, the system can safely ask the user to enter information when automatic extraction is uncertain.

When the manual form is submitted, the code rejects a blank merchant and rejects an amount that cannot be parsed as a positive number. A valid entry is categorised automatically from the merchant text, saved through DatabaseHelper and followed by a refresh callback. This path remains useful for cash expenses and unsupported notifications.

6.7 Automated Category Assignment

The current application assigns categories through a merchant mapping. The merchant text is normalised to lowercase and checked against predefined keywords. When several keywords match, the longest matching keyword is selected. If none matches, the result is Other.

This deterministic approach is easy to test. A known merchant can be given a fixed expected category, and an unknown merchant can be checked for the Other result. It also avoids the need for a training dataset in the current prototype.

However, the mapping has limited coverage. New merchants, spelling variations and ambiguous names can lead to an incorrect category or Other. The class name and project description refer to Naive Bayes, but the runtime code does not calculate Naive Bayes probabilities.

The implementation should therefore be documented accurately. Future work can introduce a trained probabilistic classifier while preserving the same category interface. The replacement should be evaluated against the current mapping so that the benefit of the change can be measured.

The name NaiveBayesClassifier is retained in the source, but the method does not calculate Bayes probabilities. It normalises the input text, filters a map of known keywords by substring match, selects the longest key and returns its stored label. The report therefore describes the current behaviour rather than the class name alone.

6.8 Transaction History and Monthly Summaries

Transaction history provides the user with a chronological record of saved expenses. Each entry includes the fields required to understand the transaction. The history is also an important diagnostic tool because users can inspect whether automatic capture and manual entry have produced the expected records.

Monthly summaries aggregate transactions into calendar-month totals. This provides a higher-level view than the history list. It also creates the input data required by the Linear Regression predictor.

The accuracy of a monthly summary depends on all earlier processing stages. Missing transactions lower the total, duplicate transactions raise it, and incorrect timestamps can place values in the wrong month. Therefore, the summary screen should be treated as a derived view of the transaction database.

Future versions can provide category-wise monthly summaries or additional charts, but these should be added only after the underlying data model and requirements are extended. The current project focuses on transaction history and monthly totals.

The transaction history displays stored rows in reverse creation-time order. Each row is represented by merchant, formatted rupee amount, category, date and time. The same stored values underpin monthly totals and predictions, so a duplicate or incorrectly parsed value can affect multiple screens.

The database query for history sorts by the creation timestamp in descending order, so newly recorded transactions appear first. The interface formats dates as day-month-year text and displays the time in a 12-hour clock format using the device locale. The monthly summary uses a different query and orders its aggregates chronologically. One important implementation detail concerns deletion: the helper identifies a row by merchant, amount and category instead of the table's unique id. If two genuine transactions share all three values, one delete action may remove both. Using the primary key for deletion would make the operation unambiguous. This matters because any change to stored records also changes the monthly totals and therefore may change the next prediction.

6.9 Spending Prediction Display

The prediction display communicates the Linear Regression estimate for the next month. It should identify the forecast period and clearly distinguish the estimate from actual spending. This distinction helps prevent users from interpreting a model output as a historical fact.

The prediction depends on the monthly totals stored in SQLite. If the user has too little history, the predictor may return no value. This is preferable to presenting a false sense of precision when the model has insufficient observations.

The supplied code uses a month index based on the position of values in the list. It calculates slope and intercept from the historical sequence and then evaluates the fitted line at the next index. The returned value is constrained to be non-negative.

The prediction display can be improved in future versions by showing historical values beside the forecast. Such a chart would make the trend easier to understand, but it should still avoid implying confidence intervals unless those have actually been calculated.

The prediction display uses a nullable result. When no monthly data exists, the screen shows that there is not enough data; otherwise it displays the estimated next-month amount and recent monthly values. The current predictor is a baseline trend calculation and no observed test-set error is supplied.

The graph uses the latest available monthly totals, up to six observations, and appends the estimated next value when a prediction exists. This compact view is useful for a quick trend comparison, but a single historical observation cannot show whether expenditure is increasing or decreasing. The current aggregation query also omits empty calendar months, so a future version should label points with the actual month rather than rely only on sequential positions. The forecast point should be visibly different from recorded amounts, and the displayed period should identify which month is being estimated. A clear empty-history state is preferable to a fabricated value, especially for new users who have not yet accumulated enough records.

6.10 Budget Recommendation: Implementation Status

The earlier report described a Budget Recommendation screen with monthly-income input, average historical spending and category-oriented guidance. A direct inspection of the supplied project ZIP does not locate that screen or its supporting implementation in the current Kotlin sources.

The current database helper creates an expenses table containing merchant, amount, category and created_at. It does not define an income table or a persisted monthly-income preference. MainActivity.kt routes only to Home, Transactions, Add Expense, AI and Prediction. Therefore, the budget recommendation flow cannot be described as implemented in this delivered version.

To complete the feature, a future revision would need to define the user input, save it reliably, establish transparent category-level rules, add the missing interface and write tests for blank income, zero income, empty history, overspending categories and recalculation after a transaction is deleted.

CHAPTER 7 – TESTING AND RESULTS 7.1 Functional Testing Strategy

Functional testing checks whether the application performs the operations described by its requirements. The project material identifies notification processing, manual expense entry, parsing, duplicate handling, SQLite storage, category assignment, transaction history and prediction generation as important functional areas.

Functional testing differs from machine-learning evaluation. A functional test can verify that a category is returned or that a prediction screen is displayed, but it cannot establish classification accuracy or forecast error. Those metrics require labelled or observed outcomes and a defined evaluation protocol.

The testing strategy therefore begins with input-path tests, continues through data processing and persistence, and ends with presentation checks. Each test should specify an input scenario, expected behaviour and evidence available from the prototype.

The source material reports functional testing but does not provide per-case numerical success rates. The expanded report retains that limitation rather than inventing test statistics.

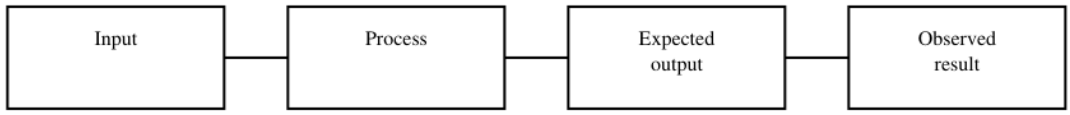


Figure 8. Functional testing workflow.

7.2 Test Environment and Test Data Principles

A useful test environment should include an Android device or emulator capable of running the application, notification access configuration, the local SQLite database and the supplied application screens. The exact device and SDK configuration were not specified in the source material and therefore are not invented here.

Test data should cover both normal and abnormal inputs. Normal cases include supported payment notifications, valid manual expenses and known merchant names. Abnormal cases include blank notification text, missing amounts, invalid numeric strings, unknown merchants and repeated events.

Forecast testing should use several monthly totals rather than a single value because the Linear Regression component requires a sequence. The future quantitative experiment should also keep evaluation months chronologically separate from the history used to construct the model.

Testing evidence should be reproducible. Screenshots can demonstrate visible results, while database inspection and controlled input cases can demonstrate persistence and duplicate handling. A future formal study should record test data and outcomes in a structured table.

The cases in this chapter specify expected functional behaviour, not a recorded formal test run. The supplied ZIP includes only basic example test scaffolding and no detailed execution log. Results are therefore stated as implementation evidence and expected outcomes; no pass rate, execution-time claim or model-quality percentage is invented.

The parser should be tested independently from Android notification delivery so that a failure can be tied to a specific input. Positive cases can include rupee-symbol values with commas or decimals and the supported Rs and INR prefixes, paired with merchant phrases beginning with "to" or "at". Negative cases should include blank text, a currency amount without a supported merchant phrase, unsupported currency notation and messages containing unrelated numbers. Duplicate tests should use controlled timestamps to check behaviour both inside and outside the two-minute window. Predictor tests should cover empty history, one month, two months, rising totals and falling totals. These are recommended test cases for a reproducible validation plan; they are not represented as completed test executions because the ZIP contains no detailed test run log.

7.3 Functional Test Cases and Expected Outcomes

Test Case 1 verifies supported notification processing. A readable notification containing a supported merchant and amount is supplied, and the expected result is that the listener passes the content to parsing and the transaction can proceed through the workflow.

Test Case 2 verifies manual entry. The user supplies a valid merchant and amount through the Add Expense screen. The expected result is that the transaction is validated, stored and visible in history. Test Case 3 verifies duplicate handling by repeating a recent merchant-and-amount combination.

Test Case 4 verifies local persistence by saving a transaction and reopening the history screen. The record should remain available. Test Case 5 verifies monthly aggregation and prediction by providing sufficient historical data and checking that a monthly total and estimated future value are displayed.

These cases demonstrate functional behaviour but do not produce machine-learning accuracy measures. They should therefore be interpreted as prototype validation rather than experimental evidence of model performance.

7.4 Boundary and Error Testing

Boundary testing checks cases close to the limits of the expected input. For amount parsing, examples should include whole numbers, decimal values, comma-separated values and unsupported numeric formats. The parser should reject values that cannot be converted safely.

Notification tests should include blank text, messages without currency amounts, messages containing multiple numbers and messages whose merchant pattern does not match the supported expression. The expected behaviour is to avoid creating an uncertain transaction.

Duplicate tests should include two identical recent events, two identical merchant-and-amount events separated by a longer interval and two legitimate purchases with the same merchant and amount. These cases help reveal the trade-off created by a time-window heuristic.

User-interface error testing should also verify missing income on the budget screen and insufficient history for prediction. The application should provide a meaningful prompt rather than displaying a misleading completed result.

7.5 Usability-Oriented Checks

Usability testing focuses on whether a user can understand and complete routine tasks. The main tasks are granting notification access, entering an expense manually, reviewing history, viewing monthly spending and accessing the budget recommendation screen.

The interface should make the difference between actual and estimated values obvious. A user who sees a forecast should be able to identify that it is model-generated. Similarly, category labels should be readable and consistent across history and summaries.

Manual entry should require as few steps as practical while still validating the required information. Notification permission should be explained before the user is directed to Android settings. These details affect whether automation is perceived as helpful or intrusive.

The supplied screenshots provide visual evidence of the prototype’s main screens. A future usability study could measure task completion time, user errors and satisfaction, but such measurements are not present in the source material and are not claimed here.

Usability review should include the first-launch name prompt, notification permission explanation, empty-history state, invalid manual amount, uncategorised merchant and no-history prediction state. These checks focus on whether the app gives a clear next action and avoids implying that a missing or estimated value is a confirmed transaction.

The review should test recovery as well as the successful path. If notification access is denied or later revoked, the user should still be able to record an expense manually. Once the first-launch name screen is completed, the saved name should remain after the application is closed and reopened and the setup screen should not reappear. Empty transaction history and unavailable predictions should produce clear messages instead of unexplained blank areas. Controls should have understandable labels, rupee amounts should use a consistent format and forecast values should look different from confirmed transactions. The app should also make it clear that notification access is an Android permission and does not mean the application has direct access to a bank account.

7.6 Results Supported by the Prototype

The current source supports an implementation-based assessment of the main expense-tracking path. The ZIP contains Kotlin code for payment-notification filtering, transaction parsing, duplicate checking, local SQLite persistence, merchant keyword categorisation, history retrieval and Linear Regression-based next-month estimation.

The evidence should not be overstated. The uploaded material does not include a labelled classification dataset, a trained probabilistic model, a controlled forecasting test set or measured accuracy, precision, recall, F1-score, MAE or RMSE results. The class named NaiveBayesClassifier uses a static keyword map, and income-based budget recommendations are not implemented in the current source snapshot.

Table 9. Prototype evidence and implementation boundaries

Component	Current status	Evidence / boundary
Notification capture	Present	NotificationReaderService filters and parses readable events after permission.
Manual entry	Present	Add Expense validates merchant and a positive numeric amount.
Local storage/history	Present	DatabaseHelper inserts, retrieves, deletes and aggregates expense rows.
Categorisation	Present; rule-based	Static keyword map in NaiveBayesClassifier; not a trained probability model.
Forecasting	Present; unvalidated	LinearRegressionPredictor returns an estimate; no measured error results supplied.
Budget recommendations	Not in supplied ZIP	Earlier screenshot is a design reference, not evidence of a working current screen.

These results support the conclusion that the prototype integrates a useful local expense workflow. They do not establish experimental model performance or confirm unimplemented budget behaviour.

The strongest evidence available for this revision is implementation evidence: the ZIP contains the relevant Kotlin classes and event handlers, and the supplied screenshots illustrate the visible application screens. The ZIP does not include a labelled classification dataset, a controlled forecasting benchmark or a detailed test execution log. For stronger evidence, each functional case should record the input, expected outcome, observed outcome, device/build details and pass/fail status. Classification should be measured on merchant descriptions that were not used to design the mapping or train a future model. Forecasting should be evaluated against later observed monthly totals. This keeps interface and integration findings separate from machine-learning performance claims.

7.7 Future Quantitative Evaluation Plan

A future classification evaluation should collect transaction descriptions and verified category labels. The dataset should represent the categories supported by the application and include common and less familiar merchant descriptions. It should be divided into development and evaluation portions.

The current merchant-keyword mapping can serve as a baseline. A trained Multinomial Naive Bayes model can then be trained on the same development data and evaluated on the same held-out examples. Accuracy, precision, recall and F1-score can be reported, with a confusion matrix showing category-level errors.

For forecasting, monthly transaction totals should be collected over a sufficiently long period. The model should be trained on earlier months and evaluated on later months. MAE and RMSE can then quantify the difference between predicted and observed expenditure.

These experiments would provide the evidence needed to determine whether a trained classifier or a more advanced forecasting method materially improves the prototype. Until such experiments are performed, the current report remains intentionally conservative.

CHAPTER 8 – CONCLUSION AND FUTURE SCOPE

8.1 Conclusion

The AI-Based Smart Expense Management System demonstrates how transaction capture, local storage and basic analytics can be combined in an Android application. In the supplied source, supported notifications and manual entries can be converted into a shared expense record, stored in SQLite, categorised using merchant keywords and summarised by month.

The Linear Regression component provides a simple next-month estimate from the available monthly totals. It includes explicit handling for empty data and a one-month baseline, but the project material contains no formal forecasting error results. Similarly, the current category implementation is deterministic keyword matching rather than a trained Naive Bayes model.

The title includes Budget Recommendations, but the current source ZIP does not contain an income-entry screen or budget recommendation algorithm. That part remains planned scope. This distinction keeps the conclusion aligned with the implementation rather than treating the earlier screenshot as proof of functioning code.

The delivered application combines notification filtering, parsing, manual entry, local persistence, merchant-keyword categorisation, monthly aggregation and Linear Regression estimation. The implementation is useful as an academic prototype because these functions are connected through a shared local data model. The planned budget-recommendation capability is not part of the supplied source snapshot and remains future work.

8.2 Project Contributions

The first contribution is the dual input path: a readable supported payment notification can be captured automatically, while manual entry covers cash expenses, unsupported providers and extraction failures. Both paths ultimately save merchant, amount, category and timestamp data in the same local database.

The second contribution is local transaction organisation. The SQLite helper supports creation, retrieval, deletion, recent duplicate checks and monthly aggregation. The interface reuses the same saved records for the dashboard, history and prediction views.

The third contribution is lightweight merchant-keyword categorisation and a simple regression baseline. These components are small and understandable enough for a mini-project, but they should be described according to their actual code. Income-based budget recommendations and a trained Naive Bayes classifier remain future enhancements.

The strongest contributions are integration and clear fallback behaviour: a supported notification can be captured automatically, a failed or unsupported capture can be replaced by manual entry, both routes create the same type of saved record, and the record can be reused for history, summaries and a future-spending estimate. The project does not claim a newly trained classifier.

8.3 Limitations

The main technical limitation is notification compatibility. The listener can process only content exposed by Android notifications, and the parser depends on currency patterns and merchant phrases such as “to” or “at”. An unsupported message can be missed, while a false positive remains possible despite filtering.

The classifier uses a fixed keyword map. It cannot learn new merchant patterns, evaluate confidence or adapt to user corrections. The Linear Regression predictor extrapolates a straight trend and may be unstable when history is short or when expenditure varies irregularly. No formal benchmark metrics were supplied.

A further scope limitation is that the current source ZIP does not implement the income-based budget recommendation feature depicted in the earlier report screenshot. A fully functional version would need income storage, recommendation logic, interface integration and test cases.

A further scope limitation is the gap between the project title/earlier screenshot and the supplied Kotlin code: income-based budgeting is not implemented in the ZIP provided for this revision. To complete that part, the application would need an income input and persistence mechanism, a defined recommendation formula, a corresponding screen and functional tests.

Several constraints follow from the specific code. The parser accepts a limited set of currency prefixes and looks for merchant phrases beginning with "to" or "at", so different notification wording may be ignored. The category map is a fixed list of substring rules and cannot learn from user corrections. Duplicate detection compares exact merchant text and amount in a recent time window; format differences can bypass it, while legitimate repeat purchases can be mistaken for duplicates. Monthly aggregation omits calendar months with no stored rows, which can affect the regression sequence. Finally, local SQLite persistence does not itself provide encryption, and debug logging should be reviewed before production release. These are concrete engineering limitations that can guide the next revision and testing plan.

8.4 Future Enhancements

Future work can expand notification parsing to support more message formats and languages. A stronger parser can use a structured set of templates and a more comprehensive validation layer. Duplicate detection can also incorporate timestamps and transaction references where available.

The category module can be upgraded to a trained Multinomial Naive Bayes classifier using a labelled transaction dataset. The existing keyword mapping can remain as a baseline or fallback. Performance should be reported using accuracy, precision, recall, F1-score and a confusion matrix.

Forecasting can be improved after establishing a reliable historical dataset. Alternative models can be compared with the current Linear Regression baseline using chronological evaluation and MAE and RMSE. The application can also provide clearer historical-versus-forecast visualisations.

Budget guidance can be extended with configurable savings goals, category limits and anomaly alerts. Security improvements such as stronger local protection and explicit data-retention controls can also be considered for a production-oriented version.

Future work can implement the budget module with explicit income storage, transparent category-limit rules and user-editable recommendations. Separately, a true Multinomial Naive Bayes implementation would require labelled examples and probability estimation. Both additions should be tested independently so that design proposals are not confused with present functionality.

A sensible implementation order is to improve data quality before adding more intelligence. First, expand notification templates and add repeatable parser tests. Next, use a stable transaction id for deletion and stronger duplicate handling where the source provides a reference. Then generate continuous calendar-month series and evaluate regression predictions against later observations. After that, build a labelled merchant dataset and compare a trained Multinomial Naive Bayes model with the current keyword baseline. The budget module can follow with persisted income, transparent calculations, editable category limits and tests for missing or unrealistic inputs. This sequence reduces the risk of generating sophisticated-looking guidance from incomplete or duplicated records.

8.5 Final Summary

The project provides a practical foundation for a smart expense-management application. The supplied source demonstrates notification filtering and parsing, manual expense entry, local persistence, merchant-keyword categorisation, transaction history, monthly aggregation and a Linear Regression-based next-month estimate.

The implementation should not be presented as more advanced than the code supports. The class named NaiveBayesClassifier is rule based, the forecasting model has no reported measured error, and the budget recommendation screen shown in an earlier report version is absent from the current ZIP. These gaps are documented as future work rather than hidden behind unsupported claims.

The next development stage should add a tested income-based budget module, create a labelled transaction dataset, implement or remove the intended Naive Bayes classifier claim, and evaluate prediction quality chronologically. The current prototype is the starting point for that work.

The prototype is best understood as a working foundation for personal transaction tracking, not as a production-ready financial adviser. Its most defensible contribution is the integration of supported notification capture with manual entry and shared local analytics. The supplied code and screenshots make the current behaviour inspectable, while the documented limitations identify what must be built or measured next. In the final project presentation, the team should distinguish fixed merchant rules from a trained Naive Bayes model, estimated spending from actual totals, and the planned budget recommendation from screens that are present in the current ZIP. Accurate scope statements strengthen the technical credibility of the report.

REFERENCES

[1] A. McCallum and K. Nigam, “A Comparison of Event Models for Naive Bayes Text Classification,” AAAI-98 Workshop on Learning for Text Categorization, 1998, pp. 41–48.

[2] C. D. Manning, P. Raghavan and H. Schütze, Introduction to Information Retrieval, Cambridge University Press, 2008.

[3] G. James, D. Witten, T. Hastie and R. Tibshirani, An Introduction to Statistical Learning: With Applications in R, 2nd ed., Springer, 2021.

[4] R. J. Hyndman and G. Athanasopoulos, Forecasting: Principles and Practice, 3rd ed., OTexts, 2021.

[5] Android Developers, “NotificationListenerService,” Android API Reference. Accessed October 2026.

[6] Android Developers, “Save data using SQLite,” Android Developers documentation. Accessed October 2026.

The references above are the principal works and technical documentation identified in the supplied project material.

[7] JetBrains, “Kotlin Documentation.” Accessed October 2026.

[8] Android Developers, “Jetpack Compose,” Android Developers documentation. Accessed October 2026.

[9] R. J. Hyndman and A. B. Koehler, “Another look at measures of forecast accuracy,” International Journal of Forecasting, vol. 22, no. 4, pp. 679–688, 2006.

[10] F. T. Liu, K. M. Ting and Z.-H. Zhou, “Isolation Forest,” in Proceedings of the 8th IEEE International Conference on Data Mining, 2008, pp. 413–422.

The reference list is retained in a conventional academic format. Access dates and web addresses should be checked against the institution’s preferred citation style if required.

APPENDIX A – APPLICATION SCREENSHOTS



Figure 9. Home dashboard showing total expenses and a quick summary.

The dashboard summarises locally stored expense records and provides entry points to the other current screens. The visible values are illustrative application records, not test-set results. The current source reloads the transaction list and monthly totals from SQLite when the activity resumes or receives the transaction-refresh broadcast.

APPENDIX A – APPLICATION SCREENSHOTS



Figure 10. Home dashboard showing category distribution and monthly spending information.

This screenshot provides a second view of the dashboard's summary information. Category totals depend on the assigned merchant category and on the completeness of saved records. Because categories currently come from a predefined keyword map, unknown or ambiguous merchant names can be placed in Other.

APPENDIX A – APPLICATION SCREENSHOTS

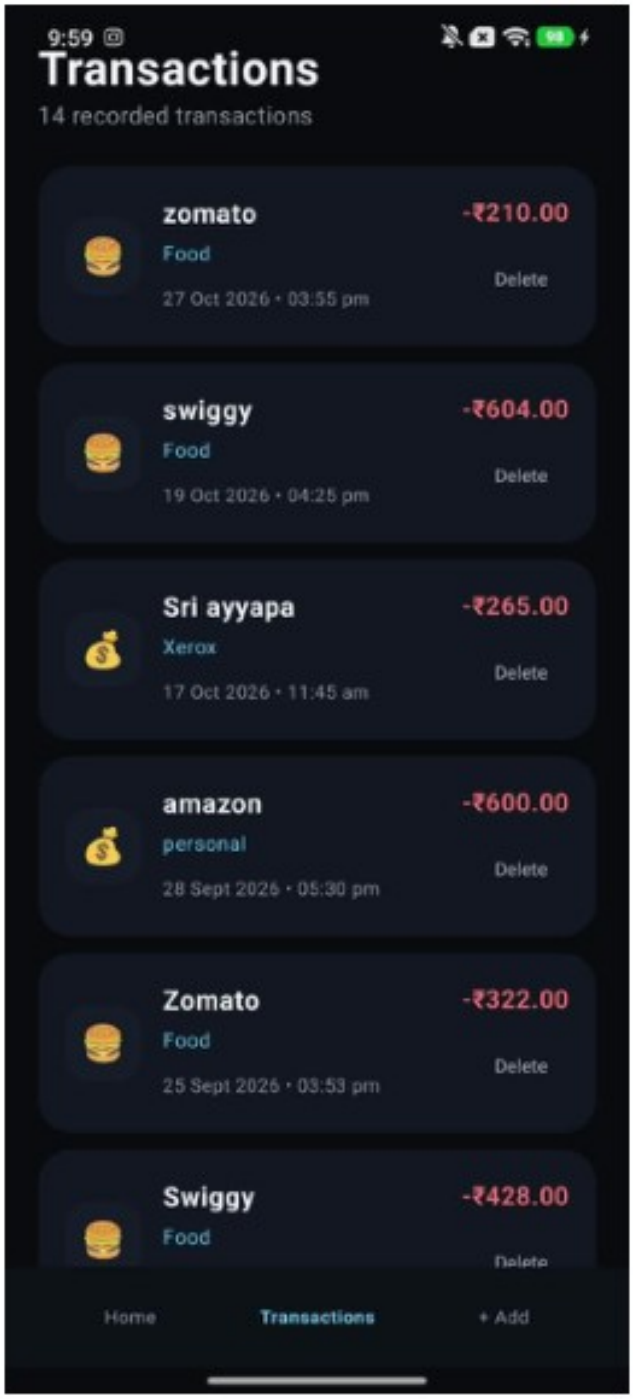


Figure 11. Transaction history with merchant names, categories, amounts and timestamps.

The history screen presents expense records read from the local database, including merchant, amount, category and formatted date/time. A missing transaction may result from an unsupported notification, an unsuccessful parse, disabled notification access or a payment that needs to be entered manually.

APPENDIX A – APPLICATION SCREENSHOTS

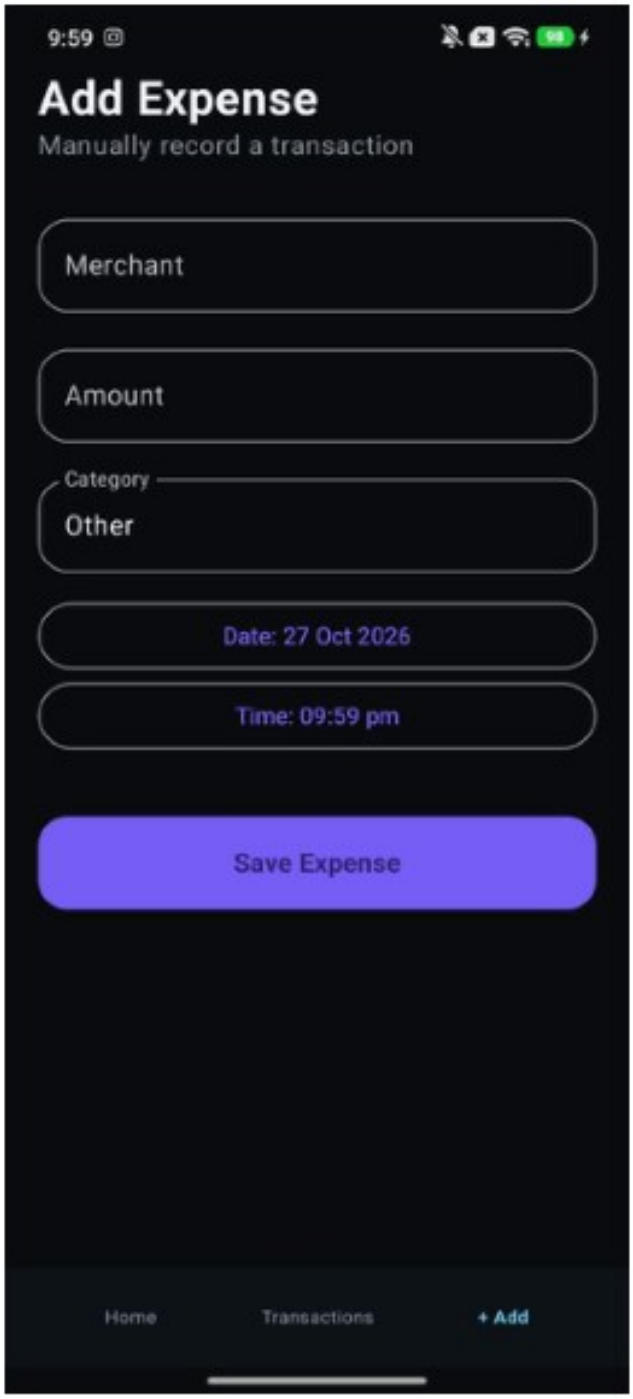


Figure 12. Add Expense interface for manual transaction entry.

The manual form is the recovery path for cash purchases and notifications that cannot be parsed. The current code requires a non-empty merchant and a valid positive amount, assigns a category from the merchant text, stores the record in SQLite and refreshes the visible data.

APPENDIX A – APPLICATION SCREENSHOTS

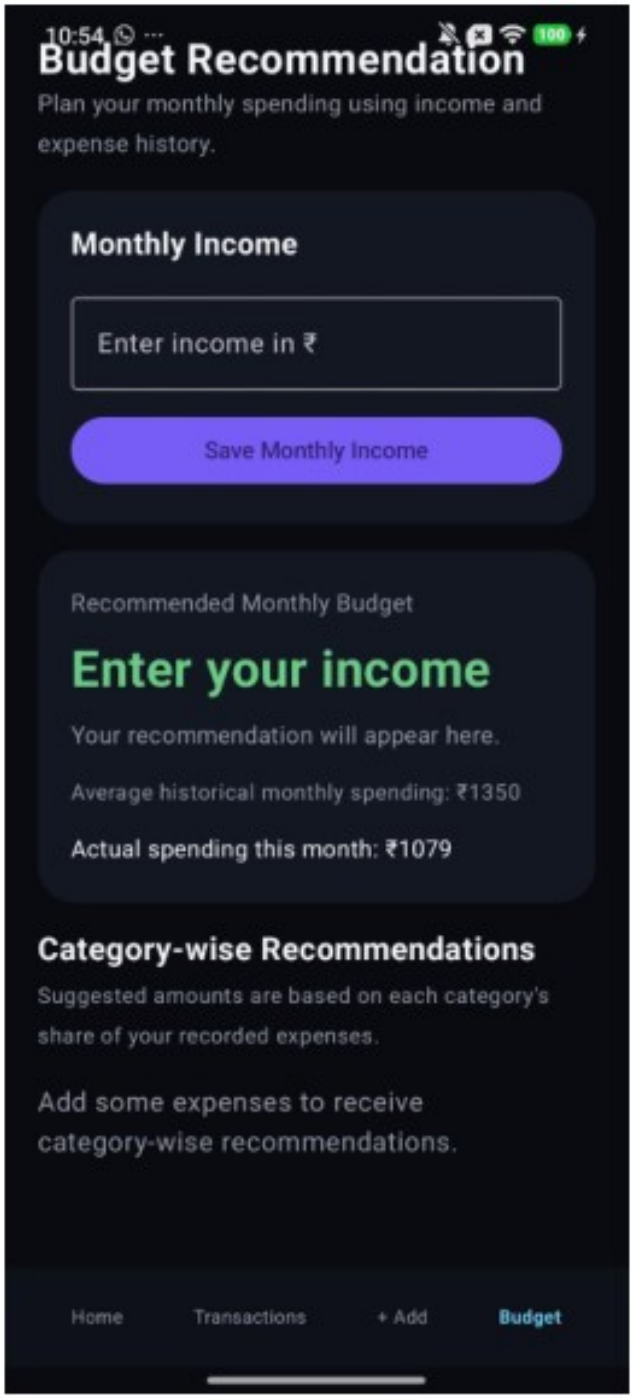


Figure 13. Budget Recommendation screen shown in the earlier supplied report (design reference only).

Alignment note: the screenshot appears in the earlier supplied report, but the current ZIP has no corresponding budget screen, income-entry field or budget recommendation calculation. It is retained here only as a design reference and must not be treated as evidence that the feature is implemented or tested in the supplied version.

APPENDIX B – IMPORTANT CODE EXTRACTS

The extracts below are taken from the Kotlin source files in the supplied AI Smart Expense project ZIP. They include the active components needed to explain notification filtering, transaction parsing, merchant categorisation, database persistence, monthly aggregation, prediction and first-launch name setup. They are intentionally selected excerpts rather than a copy of the full application.

Implementation note: ExpenseClassifier.kt is not reproduced because the supplied notification and manual-entry paths call NaiveBayesClassifier.classify(). The active class uses a predefined merchant-keyword map, not learned Bayes probabilities. No source code for an income-based budget recommendation screen or formula was found in this ZIP.

B.1 Merchant Keyword Categorisation

Source: NaiveBayesClassifier.kt

This file is the classifier called by both the notification service and the manual-entry screen. It performs case-insensitive keyword matching and picks the longest matching key. Despite the class name, the supplied implementation does not train or calculate Naive Bayes probabilities.

```
package com.miniproject.aismartexpense
object NaiveBayesClassifier {
   private val merchantCategories = mapOf(
      "swiggy" to "Food",
      "zomato" to "Food",
      "dominos" to "Food",
      "pizza" to "Food",
      "kfc" to "Food",
      "mcdonald" to "Food",
      "burger" to "Food",
      "restaurant" to "Food",
      "cafe" to "Food",
      "uber" to "Travel",
      "ola" to "Travel",
      "rapido" to "Travel",
      "metro" to "Travel",
      "irctc" to "Travel",
      "makemytrip" to "Travel",
      "air india" to "Travel",
      "electricity" to "Bills",
      "water bill" to "Bills",
      "gas bill" to "Bills",
      "broadband" to "Bills",
      "airtel" to "Bills",
      "jio" to "Bills",
      "vodafone" to "Bills",
      "amazon" to "Shopping",
      "flipkart" to "Shopping",
      "myntra" to "Shopping",
```

```
        "ajio" to "Shopping",
        "meesho" to "Shopping",
        "fashion" to "Shopping",
        "clothing" to "Shopping",
        "netflix" to "Entertainment",
        "spotify" to "Entertainment",
        "youtube" to "Entertainment",
        "prime video" to "Entertainment",
        "bookmyshow" to "Entertainment",
        "cinema" to "Entertainment",
        "gaming" to "Entertainment",
        "blinkit" to "Groceries",
        "zepto" to "Groceries",
        "bigbasket" to "Groceries",
        "dmart" to "Groceries",
        "grocery" to "Groceries",
        "supermarket" to "Groceries",
        "apollo pharmacy" to "Healthcare",
        "medplus" to "Healthcare",
        "pharmacy" to "Healthcare",
        "hospital" to "Healthcare",
        "clinic" to "Healthcare",
        "medicine" to "Healthcare"
    )
    fun classify(merchant: String): String {
        val name = merchant.trim().lowercase()
        if (name.isBlank()) return "Other"
        // Prefer the longest matching merchant keyword.
        val match = merchantCategories.entries
            .filter { name.contains(it.key) }
            .maxByOrNull { it.key.length }
        return match?.value ?: "Other"
    }
}
```

B.2 Transaction Text Parser

Source: TransactionParser.kt

The parser returns null when the amount or merchant cannot be identified. It recognises ₹, Rs/Rs. and INR amount prefixes and then tries “to Merchant” or “at Merchant” patterns. It is intentionally shown as supplied; these patterns do not cover every payment notification format.

```
package com.miniproject.aismartexpense
data class ParsedTransaction(
    val merchant: String,
    val amount: Double
)
object TransactionParser {
    fun parse(text: String): ParsedTransaction? {
        if (text.isBlank()) {
            return null
        }
        // Find amounts such as:
        // ₹350
        // ₹1,250
        // ₹350.50
```

```
        // Rs.350
        // Rs 350
        // INR 350
        val amountRegex = Regex(
            """(?i)(?:₹|rs\.?|inr)\s*([\d,]+(?:\.\d{1,2})?)"""
        )
        val amountMatch =
            amountRegex.find(text)
        if (amountMatch == null) {
            return null
        }
        val amount =
            amountMatch
                .groupValues[1]
                .replace(",", "")
                .toDoubleOrNull()
                ?: return null
        // Try "to Merchant"
        val toRegex = Regex(
            """(?i)\bto\s+([A-Za-z0-9&.'_-]+(?:\s+[A-Za-z0-9&.'_-]+){0,3})"""
        )
        val toMatch = toRegex.find(text)
        if (toMatch != null) {
            val merchant =
                toMatch
                    .groupValues[1]
                    .trim()
                    .removeSuffix(".")
                    .trim()
            if (merchant.isNotEmpty()) {
                return ParsedTransaction(
                    merchant = merchant,
                    amount = amount
                )
            }
        }
        // Try "at Merchant"
        val atRegex = Regex(
            """(?i)\bat\s+([A-Za-z0-9&.'_-]+(?:\s+[A-Za-z0-9&.'_-]+){0,3})"""
        )
        val atMatch = atRegex.find(text)
        if (atMatch != null) {
            val merchant =
                atMatch
                    .groupValues[1]
                    .trim()
                    .removeSuffix(".")
                    .trim()
            if (merchant.isNotEmpty()) {
                return ParsedTransaction(
                    merchant = merchant,
                    amount = amount
                )
            }
        }
        return null
    }
}
```

B.3 Linear Regression Predictor

Source: LinearRegressionPredictor.kt

This is the full predictor used by the dashboard and prediction screen. It returns null for no history, uses the single value as a baseline for one month, and fits a straight line for two or more monthly totals. The result is rounded and constrained to be non-negative.

```
package com.miniproject.aismartexpense
import kotlin.math.roundToInt
object LinearRegressionPredictor {
   fun predict(values: List<Double>): Double? {
      // No historical data means no prediction.
      if (values.isEmpty()) {
         return null
      }
      // With one month, use its total as a baseline estimate.
      if (values.size == 1) {
         return values.first()
            .coerceAtLeast(0.0)
            .roundToInt()
            .toDouble()
      }
      // Use linear regression when at least two months exist.
      val n = values.size.toDouble()
      val xValues = values.indices.map {
         (it + 1).toDouble()
      }
      val sumX = xValues.sum()
      val sumY = values.sum()
      val sumXY = xValues.zip(values).sumOf {
         it.first * it.second
      }
      val sumXSquare = xValues.sumOf {
         it * it
      }
      val denominator =
         n * sumXSquare - sumX * sumX
      if (denominator == 0.0) {
         return values.last()
            .coerceAtLeast(0.0)
            .roundToInt()
            .toDouble()
      }
      val slope =
         (n * sumXY - sumX * sumY) / denominator
      val intercept =
         (sumY - slope * sumX) / n
      val nextX = values.size + 1.0
      return (intercept + slope * nextX)
         .coerceAtLeast(0.0)
         .roundToInt()
         .toDouble()
   }
}
```

B.4 Notification Payment Filter

Source: NotificationReaderService.kt — isPaymentNotification()

The filter requires both a currency amount and a payment-related keyword, while rejecting words commonly associated with incoming funds, refunds, cashback, offers and reminders. This reduces false positives but is a heuristic rather than a complete transaction classifier.

```
private fun isPaymentNotification(
    text: String
): Boolean {
    val lowerText = text.lowercase()
    // Must contain a money amount
    val amountRegex = Regex(
        """(?i)(₹|rs\.?|inr)\s*[\d,]+(?:\.\d{1,2})?"""
    )
    val hasAmount =
        amountRegex.containsMatchIn(text)
    if (!hasAmount) {
        return false
    }
    /*
     * Words commonly found in outgoing
     * payment notifications.
     */
    val paymentKeywords = listOf(
        "paid",
        "payment",
        "payment successful",
        "payment completed",
        "debited",
        "debit",
        "sent",
        "transferred",
        "transfer successful",
        "upi",
        "transaction"
    )
    val hasPaymentKeyword =
        paymentKeywords.any {
            lowerText.contains(it)
        }
    if (!hasPaymentKeyword) {
        return false
    }
    /*
     * Ignore notifications that are usually
     * NOT expenses.
     */
    val ignoredKeywords = listOf(
        "received",
        "credited",
        "credit alert",
        "cashback",
        "refund",
        "refunded",
        "offer",
        "offers",
        "discount",
        "coupon",
        "reward",
```

```
            "reminder",
            "due soon",
            "pay your bill"
        )
        val shouldIgnore =
            ignoredKeywords.any {
                lowerText.contains(it)
            }
        if (shouldIgnore) {
            return false
        }
        return true
    }
```

B.5 Notification Content and Filtering

Source: NotificationReaderService.kt — selected intake excerpt

This excerpt reads the notification title and text fields, combines them, ignores the app's own notifications and applies the payment filter. The selected lines are copied from the existing method; surrounding method structure is omitted only to keep the appendix focused.

```
override fun onNotificationPosted(
    sbn: StatusBarNotification
) {
    /*
     * Ignore notifications generated by
     * our own application.
     */
    if (sbn.packageName == packageName) {
        return
    }
    val extras =
        sbn.notification.extras
    val title =
        extras
            .getCharSequence("android.title")
            ?.toString()
            ?: ""
    val text =
        extras
            .getCharSequence("android.text")
            ?.toString()
            ?: ""
    val bigText =
        extras
            .getCharSequence("android.bigText")
            ?.toString()
            ?: ""
    val subText =
        extras
            .getCharSequence("android.subText")
            ?.toString()
            ?: ""
    /*
     * Combine all notification fields.
     */
```

```
val combinedText =
    """
    $title
    $text
    $bigText
    $subText
    """.trimIndent()
Log.d(
    TAG,
    "NEW NOTIFICATION FROM: ${sbn.packageName}"
)
Log.d(
    TAG,
    "TEXT: $combinedText"
)
/*
 * IMPORTANT:
 * Ignore everything that isn't a payment.
 */
if (!isPaymentNotification(combinedText)) {
    Log.d(
        TAG,
        "IGNORED - NOT A PAYMENT"
    )
    return
}
```

B.6 Parse Result and Error Return

Source: NotificationReaderService.kt — selected parse excerpt

This excerpt attempts to parse the filtered text and exits the event handler when required transaction details cannot be extracted. It avoids saving a record when the parser returns null.

```
Log.d(
    TAG,
    "PAYMENT NOTIFICATION DETECTED"
)
/*
 * Extract merchant and amount.
 */
val parsed =
    TransactionParser.parse(
        combinedText
    )
if (parsed == null) {
    Log.d(
        TAG,
        "PAYMENT DETECTED BUT COULD NOT EXTRACT DETAILS"
    )
    return
}
Log.d(
    TAG,
    "========== PAYMENT =========="
)
Log.d(
```

```
    TAG,
    "MERCHANT: ${parsed.merchant}"
)
Log.d(
    TAG,
    "AMOUNT: ₹${parsed.amount}"
)
```

B.7 Categorisation, Duplicate Check and Save

Source: NotificationReaderService.kt — selected save excerpt

This excerpt shows the existing code that obtains the database helper, assigns the merchant category, rejects a recent duplicate, inserts the transaction and broadcasts a UI refresh signal.

```
/*
 * Open database.
 */
val databaseHelper =
    DatabaseHelper(
        applicationContext
    )
/*
 * Automatically categorize.
 */
val category =
    NaiveBayesClassifier.classify(
        parsed.merchant
    )
Log.d(
    TAG,
    "CATEGORY: $category"
)
/*
 * Check for duplicate transaction.
 *
 * This prevents GPay + SMS from creating
 * the same expense twice.
 */
val isDuplicate =
    databaseHelper.isDuplicateRecentExpense(
        merchant = parsed.merchant,
        amount = parsed.amount
    )
if (isDuplicate) {
    Log.d(
        TAG,
        "DUPLICATE PAYMENT - IGNORED"
    )
    return
}
/*
 * Save the payment.
 */
databaseHelper.addExpense(
    merchant =
        parsed.merchant,
```

```
            amount =
                parsed.amount,
            category =
                category
        )
        Log.d(
            TAG,
            "PAYMENT SAVED TO DATABASE"
        )
        /*
         * Tell MainActivity to refresh
         * the transaction history.
         */
        val updateIntent =
            Intent(
                "com.miniproject.aismartexpense.NEW_TRANSACTION"
            )
        updateIntent.setPackage(
            applicationContext.packageName
        )
        applicationContext.sendBroadcast(
            updateIntent
        )
        Log.d(
            TAG,
            "UI UPDATE SIGNAL SENT"
        )
        Log.d(
            TAG,
            "============================="
        )
    }
```

B.8 Insert and Duplicate Detection

Source: DatabaseHelper.kt — addExpense() and isDuplicateRecentExpense()

The insert records a creation timestamp in milliseconds. Duplicate detection queries for the same merchant and amount within the previous two minutes. This exact-match heuristic is useful for repeated notification events but may conflate two legitimate purchases with the same merchant and amount.

```
fun addExpense(
    merchant: String,
    amount: Double,
    category: String
) {
    val db = dbHelper.writableDatabase
    val values = ContentValues().apply {
        put("merchant", merchant)
        put("amount", amount)
        put("category", category)
        put("created_at", System.currentTimeMillis())
    }
    db.insert(
        "expenses",
        null,
        values
```

```
    )
    db.close()
}
fun isDuplicateRecentExpense(
    merchant: String,
    amount: Double
): Boolean {
    val db = dbHelper.readableDatabase
    val twoMinutesAgo =
        System.currentTimeMillis() - (2 * 60 * 1000)
    val cursor = db.query(
        "expenses",
        arrayOf("id"),
        "merchant = ? AND amount = ? AND created_at >= ?",
        arrayOf(
            merchant,
            amount.toString(),
            twoMinutesAgo.toString()
        ),
        null,
        null,
        null,
        "1"
    )
    val exists = cursor.moveToFirst()
    cursor.close()
    db.close()
    return exists
}
```

B.9 Monthly Spending Aggregation

Source: DatabaseHelper.kt — getMonthlyExpenseTotals()

The query aggregates saved records into calendar-month totals using the local-time interpretation of created_at, then orders the groups chronologically. Only months present in the grouped query are returned; there are no generated zero-spend months.

```
fun getMonthlyExpenseTotals(): List<Double> {
    val totals = ArrayList<Double>()
    val db = dbHelper.readableDatabase
    val cursor = db.rawQuery(
        """
        SELECT
            strftime(
                '%Y-%m',
                datetime(
                    created_at / 1000,
                    'unixepoch',
                    'localtime'
                )
            ) AS month,
            SUM(amount) AS total
        FROM expenses
        GROUP BY month
        ORDER BY month ASC
        """.trimIndent(),
        null
```

```
        )
        while (cursor.moveToNext()) {
            val total =
                cursor.getDouble(
                    cursor.getColumnIndexOrThrow("total")
                )
            totals.add(total)
        }
        cursor.close()
        db.close()
        return totals
    }
```

B.10 SQLite Schema and Upgrade

Source: DatabaseHelper.kt — ExpenseDatabaseHelper

The current database table stores an auto-increment id, merchant, amount, category and created_at. Version 2 adds created_at for an older database when needed. The database is local and no remote sync layer is present in the supplied project.

```
private class ExpenseDatabaseHelper(
    context: Context
) : android.database.sqlite.SQLiteOpenHelper(
    context,
    "expenses.db",
    null,
    2
) {
    override fun onCreate(db: android.database.sqlite.SQLiteDatabase) {
        db.execSQL(
            """
            CREATE TABLE expenses (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                merchant TEXT NOT NULL,
                amount REAL NOT NULL,
                category TEXT NOT NULL,
                created_at INTEGER NOT NULL
            )
            """.trimIndent()
        )
    }
    override fun onUpgrade(
        db: android.database.sqlite.SQLiteDatabase,
        oldVersion: Int,
        newVersion: Int
    ) {
        if (oldVersion < 2) {
            // Existing database is preserved.
            // created_at is added only if required.
            try {
                db.execSQL(
                    "ALTER TABLE expenses ADD COLUMN created_at INTEGER NOT NULL DEFAULT 0"
                )
            } catch (_: Exception) {
                // Column already exists.
            }
        }
    }
}
```

B.11 One-Time Name Setup Persistence

Source: MainActivity.kt — SharedPreferences excerpts

The activity reads the saved name during startup. If the value is blank, it shows NameSetupScreen; saving a name writes it to app_preferences and updates the state. Subsequent launches read the saved value and skip the setup screen.

```
preferences = getSharedPreferences(
   "app_preferences",
   Context.MODE_PRIVATE
)
userNameState.value =
   preferences.getString("user_name", "") ?: ""
            onNameSaved = { enteredName ->
               preferences.edit()
                  .putString("user_name", enteredName)
                  .putBoolean("name_setup_completed", true)
                  .apply()
               userNameState.value = enteredName
            }
```

B.12 Manual Expense Validation

Source: MainActivity.kt — Add Expense save action

Manual entry rejects a blank merchant and an amount that is missing, not numeric or less than/equal to zero. It then categorises the merchant through NaiveBayesClassifier, calls DatabaseHelper.addExpense and clears the input fields.

```
   onClick = {
      val amountValue =
         amount.toDoubleOrNull()
      if (merchant.trim().isEmpty()) {
         error = "Enter a merchant name."
      } else if (
         amountValue == null || 
         amountValue <= 0
      ) {
         error = "Enter a valid amount."
      } else {
         val detectedCategory =
            NaiveBayesClassifier.classify(merchant.trim())
         val db = DatabaseHelper(context)
         db.addExpense(
            merchant.trim(),
            amountValue,
            detectedCategory
         )
         merchant = ""
         amount = ""
         category = "Other"
         error = ""
         onExpenseAdded()
      }
```

B.13 Delete Expense Action and Persistence

Source: MainActivity.kt and DatabaseHelper.kt — delete action

The transaction history exposes a Delete control. The UI converts the formatted rupee amount back to a number, calls the database helper with merchant, amount and category, then refreshes the list. The helper uses a parameterised SQLite delete statement. Because it matches these three fields rather than a unique row id, identical records may be deleted together; primary-key deletion would be safer in a future revision.

```
val db = DatabaseHelper(context)
val amount = expense.amount
   .replace("₹", "")
   .replace(",", "")
   .toDoubleOrNull()
   ?: 0.0

db.deleteExpense(
   expense.merchant,
   amount,
   expense.category
)
onDeleted()
fun deleteExpense(
   merchant: String,
   amount: Double,
   category: String
) {
   val db = dbHelper.writableDatabase
   db.delete(
      "expenses",
      "merchant = ? AND amount = ? AND category = ?",
      arrayOf(
         merchant,
         amount.toString(),
         category
      )
   )
   db.close()
}
```

International Multidisciplinary Conference on Current Research Trends-2026 (Virtual)

30 October 2026
Visit Our Website: https://imccrt.iikr.in/index.html
Contact Email: imccrt2026@gmail.com

IIKR

Letter of Acceptance

Abstract ID: IMCCRT-2026-2584 **Date: 14-10-2026**

Dear Sir/Mam,

Congratulations!!!

We are pleased to inform you that your abstract titled **"AI-Based Smart Expense Management System with Automated Categorization and Budget Recommendations"** has been accepted for presentation in the **13th International Multidisciplinary Conference on Current Research Trends-2026** is scheduled to take place on the 30 October 2026. To confirm your registration, kindly pay your conference fee as per the invoice given in page 2.

Please mention your Abstract ID (IMCCRT-2026-2583) when you do your payment transaction.
All accepted abstracts will be added in the conference proceeding with ISBN number.
Please submit your full paper for review on imccrt2026@gmail.com

After conference a separate email will be sent to the authors for the journal options and publication process. The article publication process time will depend on the journal in which the author's paper is processed for publication.

Thank you very much for your kind participation. Please do not hesitate to contact us if you need assistance.

Best regards,
Contact Email: **imccrt2026@gmail.com**
Website: https://imccrt.iikr.in/index.html

13th International Multidisciplinary Conference on Current Research Trends-2026 (Virtual)

30 October 2026
Visit Our Website: https://imccrt.iikr.in/index.html
Contact Email: imccrt2026@gmail.com

Payment Invoice

Invoice No: 584 **Date: 14-10-2026**

Account Name: HM PUBLISHER
Account Number: 22360210001653
Bank Name: UCO Bank
IFSC Code: UCBA0002236
Branch: Yavatmal, Maharashtra
PayPal Link: http://paypal.me/ijrise

Please select the Registration Category for Payment:
Email your Proof of Payment (Receipt) Once the Payment Done Successfully:

Conference Registration- Fees Details

Conference Fee Includes:	**Conference Fee: 3000 INR / 50 USD**
• Conference Proceeding with ISBN • Free of Cost Publication in DOI & Google Scholar Indexed Journal • Digital Certificate of Participation • Digital Certificate of Presentation	

Scan and Save the Proof of Payment (Receipt) in pdf/image and **email** to **imccrt2026@gmail.com**
Use the following emails subject: Conference Proof of Payment + Your Abstract ID+ Full Name of Presenter + Contact Number.

For Example: Conference Proof of Payment (Abstract ID: 100) JOHN+011111

NOTE:

- **Bank charges are to be paid by the participant.**
- **Please send us your payment proof BEFORE 25 October 2026**
- **Additional publication fee to be paid by the participant.**

IMCCRT-2026 Secretariat

Contact Email: imccrt2026@gmail.com
Website: https://imccrt.iikr.in/index.html